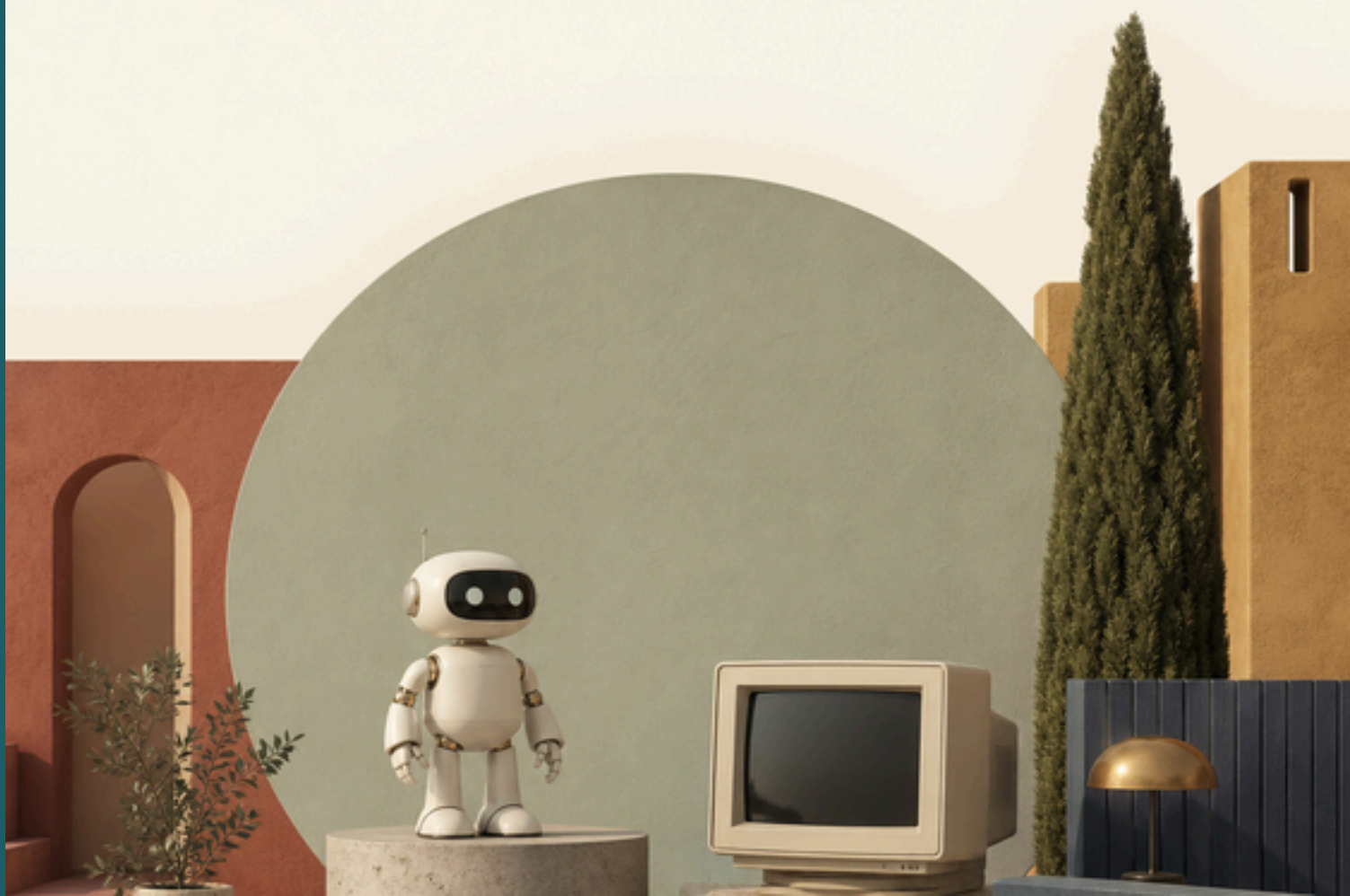
PRIME SCHOOL

Computing & Robotics

Year 7

Cambridge Lower Secondary
Student Manual



✦ PRIME BOOKS

Computing 7

Student's Book · Cambridge Lower Secondary · Ages 11–14

PUBLISHED BY

Prime Books, the publishing imprint of Prime School.
Cascais, Portugal.
primeschool.pt

EDITION

First edition, published 2026.
Impression 1.

WRITTEN AND EDITED BY

The Prime School Computing & Robotics faculty, with design and illustrations by the Prime Books studio.

ABOUT THIS BOOK

This book was written by Prime School teachers for pupils in Years 7 to 9. It follows the Cambridge Lower Secondary Computing curriculum, covering programming, robotics, networks, data and digital literacy. It is an independent publication and is not affiliated with or endorsed by Cambridge University Press & Assessment.

A NOTE ABOUT TRADEMARKS

Python, micro:bit, Microsoft, Excel, Access and Scratch are trademarks of their respective owners. They are named here for identification and educational purposes only.

WEBSITES

Web addresses were correct when this book went to press. Prime School is not responsible for the content of other websites.

STAYING SAFE

Practical tasks have been checked for hazards, but your teachers will always supervise practical work and guide you to use the internet safely.

LANGUAGE

This book is written in British English.

ISBN
978-989-0000-07-1

PAPER

FSC-certified stock from responsibly managed forests

PRINTED IN

Portugal

Contents

Introduction

Somebody had to write the software in your pocket. This year, that somebody starts to be you.

Computing is the study of how machines store information, how they process it, and how people and machines work together. It is a young subject, and an unusually practical one. Almost everything you learn in this book can be tried out within minutes of reading about it.

You already live surrounded by computing. The card reader on the 15E tram, the app that tells you when the next ferry leaves Cais do Sodré, the model that predicts whether the swell at Guincho will be worth the bus ride: each one is a program that somebody designed, wrote, tested and improved.

This book will show you how that is done, and then ask you to do it yourself.

What you will learn

Across six units you will build a foundation in four connected areas.

- **Programming.** Moving from Scratch blocks to typed Python, then on to physical computing with the micro:bit.
- **Computational thinking.** Breaking hard problems into smaller ones, spotting patterns, and planning solutions as algorithms before you write any code.
- **Networks and the internet.** How a request leaves your laptop and comes back as a web page, and how to stay safe while it happens.
- **Data and intelligence.** How computers represent pictures, sound and text as numbers, how data is modelled, and how artificial intelligence learns from examples.

How the units are built

Every unit opens with a real situation and closes with a project that solves it. In between, short bursts of explanation alternate with things for you to try. You are not expected to read a unit straight through: keep a computer beside you and stop often.

The habits matter as much as the knowledge. Plan before you type. Name things clearly. Test with values chosen to break your own work. When something fails, read the error message properly before changing anything. These habits are what separate a programmer from somebody who merely knows some Python.

A word about mistakes

Your programs will not work the first time. This is not a sign that you are bad at computing: it is what computing actually looks like, for everybody, permanently. Professional engineers spend far more time reading errors and fixing faults than they spend writing new lines.

Treat every bug as a small puzzle with a guaranteed solution, because that is exactly what it is.

✧ DID YOU KNOW?

The word **bug** for a computer fault was popularised in 1947, when engineers working on the Harvard Mark II found a moth caught in a relay. They taped the moth into the logbook and noted that they had been "debugging" the machine. The logbook, moth included, still survives.

Prime School · Computing & Robotics faculty
Cascais, 2026

How to use this book

Every unit uses the same set of features. Once you recognise them, you can navigate any page at a glance.

◆ GET STARTED & SCENARIO

Opens the unit with a real situation, usually somewhere in Portugal, and the questions it raises. Discuss these before reading on.

✦ WARM UP

A short puzzle to wake up the right part of your brain. No computer needed, and no marks at stake.

↺ DO YOU REMEMBER?

Connects the new unit to what you already learned in earlier years. Read it even if you feel confident.

▤ LEARN

The explanation itself, with worked examples, tables and code. Terms printed in **bold** are defined in the keyword strips and in the glossary.

● PRACTISE

Tasks to try immediately after reading. Attempt every one: skipping practice is the fastest way to fall behind in computing.

✧ DID YOU KNOW?

A piece of history or an odd fact. Not examined, but the sort of thing worth knowing.

→ GO FURTHER

An extra idea beyond the core material, for when you have finished and want more.

▲ CHALLENGE YOURSELF

A harder problem that pulls several ideas together. Expect to need more than one attempt.

▮ KEYWORDS

Definitions of the important terms, right where you meet them. All of them reappear in the glossary at the back.

★ FINAL PROJECT

A full page at the end of each unit: a brief, numbered milestones, a worked example to check against, and the marking grid your teacher will use.

◈ EVALUATION

A traffic-light grid. Colour green if you could teach it, amber if you need your notes, red if you want another look.

✓ WHAT CAN YOU DO?

A checklist of everything the unit covered. Tick each box only when you genuinely can do it.

Code blocks
Dark panels show Python exactly as you should type it. Comments begin with #.

Output panels
Dashed panels show what appears on screen when that code runs. Compare yours against them.

Flowcharts
Rounded box = start or stop. Slanted box = input or output. Rectangle = process. Diamond = decision.

Getting set up

Everything you need for this book is free. Work through this page once, at the start of the year, and you will not have to think about it again.

1. Getting Python onto your computer

Go to **python.org**, choose **Downloads**, and take the latest version 3 release for your operating system.

1. Run the installer you downloaded.
2. **On Windows**, tick **Add python.exe to PATH** on the first screen before you click Install. This one tick prevents a great many problems later.
3. **On macOS**, run the installer package and accept the defaults.
4. When it finishes, find **IDLE** in your applications and open it.

You should see a window with the `>>>` prompt. Type the line below and press Enter.

```
print("Prime School, ready")
```

```
Prime School, ready
```

If that worked, Python is installed correctly.

2. Keeping your work tidy

Make one folder for the whole year, with a subfolder for each unit. Save every program inside it.

```
Computing 7/
  Unit 7.1/
    fare.py
    trip_calculator.py
  Unit 7.2/
  Unit 7.3/
```

File names should use lower case letters, digits and underscores only, and must end in `.py`. Never put a space or an accented character in a Python file name.

3. Meet the micro:bit (Units 7.2 and 7.6)

The BBC micro:bit is a small circuit board with buttons, lights and sensors built in. You will program it in Python from your browser at **python.microbit.org**, so nothing needs installing.

1. Connect the micro:bit to your computer with a USB cable.
2. Open the editor in your browser and write your program.
3. Click **Send to micro:bit**, or click **Download** and drag the file onto the MICROBIT drive that appears.
4. The yellow light on the back flashes while the program transfers, then the micro:bit restarts and runs it.

Always unplug the micro:bit by ejecting the drive first, exactly as you would with a memory stick.

4. Staying safe online

- ✓ Handle boards by the edges. Static electricity from a jumper can damage the electronics.
- ✓ Never connect anything to the micro:bit pins other than the components your teacher provides.
- ✓ Save your work often. Press **Ctrl** and **S** together, every few minutes, until it becomes a reflex.
- ✓ Keep a backup of your year folder in your school cloud storage.
- ✓ Never share passwords, and never type a password into a page you reached from a link in a message.

→ IF SOMETHING WILL NOT WORK

a. Read the error message from the bottom line upwards.

b. Check capital letters, colons, brackets and quotation marks.

c. Check the indentation of every block.

d. Close IDLE completely and reopen it.

e. Ask a partner to read your screen aloud. Saying code out loud finds faults that staring at it will not.

PYTHON

Version 3, from python.org, with IDLE included

MICRO:BIT

python.microbit.org, no needed

UNIT

7.1

Coding and Programming

✓ LEARNING OUTCOMES

In this unit you will learn to:

- ✓ name the main data types used in Python and choose the right one for a value
- ✓ write programs that take input from a user and display clear output
- ✓ store values in variables and use them inside calculations
- ✓ use the arithmetic operators, including whole-number division and remainder
- ✓ convert values between data types using casting
- ✓ draw algorithms as flowcharts using the standard symbols
- ✓ predict what a flowchart will do when it contains a decision
- ✓ write selection statements with the comparison operators
- ✓ build a test plan and use it to prove a program works
- ✓ recognise syntax, runtime and logic errors, then debug them

◆ GET STARTED

Have you ever wondered what a vending machine is really thinking?

Find the drinks machine in the school hall, or picture one you have used. Talk about these questions with the person next to you.

- What does the machine need you to give it before it will do anything?
- Which buttons make it work something out on its own, such as your change?
- How does it tell you what is happening?

A vending machine is a computer program wearing a metal coat. It waits for input, stores what it is given, performs a calculation, then produces output. It only feels reliable because somebody tested it very thoroughly before it was bolted to the wall.

In this unit you will move from the coloured blocks of Scratch to **text-based programming** in Python. You will write programs that ask questions, remember answers, do sums with them and make decisions, and you will learn how to prove that your programs are right.

■ KEYWORDS

text-based programming writing a program by typing instructions as words and symbols rather than dragging ready-made blocks

Python a widely used text-based programming language, popular in schools, science and industry

✦ WARM UP

Gonçalo has written a shopping list for a school picnic at Praia do Guincho. Sort each item into one of three groups: **a whole number**, **a number with a decimal part**, or **text**.

1. The number of baguettes to buy: 12
2. The price of one baguette: €1.35
3. The name of the bakery: Padaria do Mar
4. The number of pupils travelling: 28
5. The total weight of the cool box in kilograms: 4.7
6. The bus registration: 42-BC-19

Now answer this. Item 6 contains digits, so why can it never be stored as a number?

↺ DO YOU REMEMBER?

In Year 6 you built projects in Scratch. You already know more than you think.

- An orange **variable** block such as *set score to 0* keeps a value for later.
- A blue **input** block such as *ask and wait* collects something from the user.
- A purple **output** block such as *say Hello* shows a result on the screen.
- A yellow **control** block such as *if … then* chooses between two paths.

Python has all four of these ideas. The difference is that you type them instead of dragging them.

UNIT 7.1 · TOPIC 1

From coloured blocks to typed lines

Scratch is a **block-based** language. Every instruction is a shape you drag into place, and shapes that would not make sense simply refuse to click together. That is a friendly way to start, because the language stops you making many mistakes.

Python is a **text-based** language. You type each instruction as a line of text. Nothing stops you typing something impossible, so you have to be precise. In return you get speed, power and a language used by real engineers.

The same idea in two languages

Imagine a program that greets a pupil by name. In Scratch you would drag an *ask* block, then a *say* block. In Python it is two typed lines.

```
name = input("What is your name? ")
print("Bom dia, " + name + "!")
```

```
What is your name? Matilde
Bom dia, Matilde!
```

Both versions do exactly the same job. The Python version is shorter to write once you can type it, and it fits on a single line of a printed page.

IDEA	IN SCRATCH	IN PYTHON
Show something	*say Hello*	`print("Hello")`
Ask a question	*ask What is your name? and wait*	`input("What is your name? ")`
Store a value	*set score to 0*	`score = 0`
Add to a value	*change score by 1*	`score = score + 1`
Make a choice	*if … then … else*	`if ...: ... else: ...`

Why the switch is worth it

- Text is compact. A program that fills a whole screen in Scratch may be twelve typed lines.
- Text can be searched, copied and shared as a simple file.
- Almost every professional language is text-based, so the habits transfer.
- Python reads a little like English, which makes it a kind first text language.

✧ DID YOU KNOW?

Python was released in 1991 by Guido van Rossum, a Dutch programmer. He named it after the television comedy *Monty Python's Flying Circus*, not after the snake. The snake logo arrived later, because a snake is much easier to draw than a comedy sketch.

● PRACTISE 1

Work with a partner.

1. Write down three things Scratch does that you would miss in Python.
2. Write down three things you think typed code will let you do more easily.
3. Look at the table above. Which single row do you think will be hardest to remember, and why?

UNIT 7.1 · TOPIC 2

Meeting IDLE

Python usually arrives with a small program called **IDLE**, which stands for Integrated Development and Learning Environment. IDLE gives you two ways to work, and knowing which one you are in saves a great deal of confusion.

The shell: thinking out loud

The **shell** runs one line at a time and answers immediately. You will recognise it by the three arrows, `>>>`, called the prompt. The shell is perfect for trying an idea.

```
>>> 7 * 6
42
>>> print("Olá, Cascais")
Olá, Cascais
>>> 15 + 27
42
```

The shell forgets nothing while it is open, but it saves nothing when it closes. It is a workbench, not a filing cabinet.

Script mode: writing something to keep

Script mode is a plain editor window. You type a whole program, save it as a file ending in `.py`, then run the lot in one go. Every program you hand in will be written this way.

To create, save and run a program:

1. In IDLE choose **File**, then **New File**. An empty editor window opens.
2. Type your program.
3. Choose **File**, then **Save**. Give the file a sensible name such as `fare.py`.
4. Choose **Run**, then **Run Module**, or simply press **F5**.
5. The results appear in the shell window.

Getting the case right

Python cares about capital letters. `print` is a command; `Print` is not. This single detail causes more first-week errors than anything else.

```
>>> Print("Hello")
Traceback (most recent call last):
  File "<pyshell#0>", line 1, in <module>
    Print("Hello")
NameError: name 'Print' is not defined
```

Leaving notes with comments

Anything after a `#` is a **comment**. Python ignores it completely, so comments are for the humans who read your work, including you in three weeks' time.

```
# Ferry timetable helper
# Inês, Year 7, Prime School
crossings = 14        # sailings each weekday
print(crossings)
```

KEYWORDS

IDLE the editor and shell that comes with Python

shell a window that runs one instruction at a time and shows the result straight away

script mode writing a complete program in a file so it can be saved and run again

comment a note in the code, starting with `#`, that Python ignores

---

PRACTISE 2

1. Open the IDLE shell and work out `48 * 27` without touching a calculator app.
2. In the shell, make Python print your favourite place in Portugal.
3. Open a new file, save it as `hello.py`, and make it print two lines: a greeting and today's date as text.
4. Add a comment at the top of `hello.py` giving your name and the date.
5. Deliberately type `print` with a capital `P`, run the file, and write down the exact error message you see.

UNIT 7.1 · TOPIC 3

Variables: giving values a name

A **variable** is a named box in memory. You put a value in it, and afterwards you can use the name instead of the value. When the value needs to change, you change it in one place.

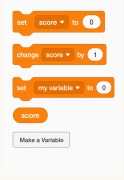
The `=` sign does not mean "equals" in the mathematical sense. It means "put the value on the right into the name on the left".

```
# Tram 28 fare calculator
passengers = 4
fare = 3.20
total = passengers * fare
print("Total fare:", total)
```

```
Total fare: 12.8
```

Four passengers at €3.20 each comes to €12.80. Python displays `12.8` because a trailing zero adds nothing to the value of a number. You will learn to format money properly later in the unit.

Rules for naming



- Start with a letter or an underscore, never a digit.
- Use letters, digits and underscores only. No spaces and no accents.

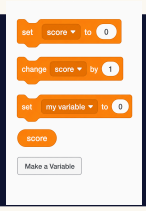
- Choose a name that says what the value is: `fare` beats `f`, and `total_cost` beats `x`.
- Remember that `fare` and `Fare` are two different variables.

NAME	ALLOWED?	WHY
`ticket_price`	Yes	clear, lower case, underscore
`2ndPrice`	No	begins with a digit
`total cost`	No	contains a space
`preço`	No	contains an accented character
`n`	Yes, but poor	legal yet tells the reader nothing

Changing what is in the box

A variable holds one value at a time. Assigning again replaces what was there.

```
lisbon_temp = 18
print(lisbon_temp)
lisbon_temp = 24
print(lisbon_temp)
```




```
18
24
```

KEYWORDS

variable a named place in memory that holds a value which can change while the program runs

assignment using = to put a value into a variable

PRACTISE 3

1. Create variables for a pupil's name, their year group and their house colour, then print all three on one line.
2. Beatriz cycles 4.5 km to school each way. Use variables to calculate and print the distance she cycles in a five-day week.
3. Explain in one sentence why `distance = distance + 2` is not nonsense, even though it would be in maths.
4. Which of these are legal variable names? `bus_number`, `3rd_place`, `Total`, `mean value`, `_count`

Input and output

A program that always does exactly the same thing is not much use. **Input** lets the user supply the values; **output** shows the result.

Talking to the user

`print()` sends text to the screen. You can pass it several items separated by commas, and Python puts a single space between them.

```
name = "Rodrigo"
house = "Atlântico"
print("Pupil:", name, "House:", house)
```

```
Pupil: Rodrigo House: Atlântico
```

Listening to the user

`input()` shows a message, waits for the user to type something and press Enter, then hands back what was typed.

```
city = input("Which city do you live in? ")
print("There is a lot to explore in " + city + ".")
```

```
Which city do you live in? Porto
There is a lot to explore in Porto.
```

Notice the space at the end of `"Which city do you live in? "`. Without it the user's typing would begin immediately after the question mark, which looks careless.

The trap that catches everybody

`input()` **always** gives you text, even when the user types digits. Adding two pieces of text joins them end to end instead of adding them up.

```
a = input("First number: ")
b = input("Second number: ")
print(a + b)
```

```
First number: 20
Second number: 22
2022
```

Python did exactly what was asked: it joined the text `"20"` to the text `"22"`. Topic 6 shows how to fix this properly.

KEYWORDS

input data given to a program by the user or by a sensor

output information the program sends out, usually to the screen

concatenation joining two pieces of text end to end with `+`

PRACTISE 4

1. Write a program that asks for a pupil's first name and their favourite subject, then prints one friendly sentence using both.
2. Write a program that asks for the name of a Lisbon tram route and prints `Route 28 runs to Campo Ourique.` with the route number the user typed.
3. Predict the output of the program below, then run it to check.

```
x = input("Type 5: ")
y = input("Type 3: ")
print(x + y)
```

4. Explain in your own words why question 3 does not print `8`.

UNIT 7.1 · TOPIC 5

Data types

Every value in Python has a **data type**. The type decides what the value means and what you are allowed to do with it.

DATA TYPE	PYTHON NAME	EXAMPLES	TYPICAL USE
String	`str`	`"Sintra"`, `"42-BC-19"`, `"7"`	names, codes, anything typed
Integer	`int`	`28`, `0`, `-6`	counts of whole things
Real	`float`	`3.20`, `-0.5`, `19.75`	money, measurements, averages
Boolean	`bool`	`True`, `False`	answers to yes or no questions

Strings are always written inside quotation marks. Single and double quotes both work, as long as you use the same one at each end.

Asking Python what it is holding

The `type()` command reports the data type of any value.

```
print(type(28))
print(type(3.20))
print(type("Sintra"))
print(type(True))
```

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

Choosing the right type

- A house number such as `14` is a count, so use an integer.
- A postcode such as `2750-642` contains a hyphen, so it must be a string.
- A temperature of `19.4` degrees has a decimal part, so use a real number.
- Whether a pupil is present is yes or no, so use a Boolean.

✧ DID YOU KNOW?

Computers store real numbers as approximations, which is why `0.1 + 0.2` prints `0.30000000000000004` in Python. It is not a bug in Python: it is the price of squeezing an infinite number line into a fixed amount of memory. For money, careful programmers work in whole cents.

■ KEYWORDS

data type the kind of value being stored, such as string, integer, real or Boolean

string a sequence of characters treated as text

integer a whole number, positive, negative or zero

real a number with a decimal part, called a `float` in Python

Boolean a value that is either `True` or `False`

● PRACTISE 5

1. State the most suitable data type for each: a pupil's surname; the number of goals scored; the price of a bus ticket; whether homework is complete; a mobile telephone number.
2. Use `type()` in the shell to check three values of your own choosing.
3. Explain why a telephone number should be stored as a string even though it looks like a number.

UNIT 7.1 · TOPIC 6

Casting: changing type on purpose

Casting converts a value from one data type to another. It is the cure for the input trap you met in Topic 4.

COMMAND	WHAT IT DOES	EXAMPLE	RESULT
`int(x)`	turns x into a whole number	`int("20")`	`20`
`float(x)`	turns x into a real number	`float("3.5")`	`3.5`
`str(x)`	turns x into text	`str(42)`	`"42"`

Fixing the adding-up problem

Wrap `input()` in `int()` and Python converts the text before storing it.

```
a = int(input("First number: "))
b = int(input("Second number: "))
print("Total:", a + b)
```

```
First number: 20
Second number: 22
Total: 42
```

Casting the other way

To join a number onto a string with `+`, the number must first become text.

```
goals = 7
print("Leonor scored " + str(goals) + " goals.")
```

```
Leonor scored 7 goals.
```

Using commas in `print()` avoids the need to cast at all, because `print` converts each item for you.

```
goals = 7
print("Leonor scored", goals, "goals.")
```

```
Leonor scored 7 goals.
```

When casting fails

`int()` can only convert text that really is a whole number.

```
>>> int("seven")
Traceback (most recent call last):
  File "<pyshell#0>", line 1, in <module>
    int("seven")
ValueError: invalid literal for int() with base 10: 'seven'
```

KEYWORDS

casting converting a value from one data type to another, for example `int("20")`

PRACTISE 6

1. Write a program that asks for two whole numbers and prints their sum, their difference and their product.
2. Duarte buys `n` pastéis de nata at €1.20 each. Ask for `n`, then print the total cost.
3. Predict what `int("4.8")` does, then try it and explain the result.
4. Rewrite `print("Age: " + age)` so it works when `age` holds the integer `12`.

UNIT 7.1 · TOPIC 7

Arithmetic operators and BIDMAS

Python uses familiar symbols for arithmetic, plus three that may be new.

OPERATOR	MEANING	EXAMPLE	RESULT
`+`	add	`9 + 4`	`13`
`-`	subtract	`9 - 4`	`5`
`*`	multiply	`9 * 4`	`36`
`/`	divide	`9 / 4`	`2.25`
`//`	integer division, whole part only	`9 // 4`	`2`
`%`	modulus, the remainder	`9 % 4`	`1`
`**`	raise to a power	`9 ** 2`	`81`

`/` always produces a real number, even when the division is exact. `10 / 2` gives `5.0`, not `5`.

Why // and % are so useful

Together they answer the question "how many whole groups, and how many left over?".

```
# Seating pupils in minibuses
pupils = 47
seats = 8
buses = pupils // seats
spare = pupils % seats
print("Full minibuses:", buses)
print("Pupils in the last bus:", spare)
```

```
Full minibuses: 5
Pupils in the last bus: 7
```

Check the arithmetic: 5 full minibuses carry 40 pupils, leaving 7. A sixth vehicle is still needed for those 7, which is exactly the kind of detail a good programmer notices.

BIDMAS

Python follows the usual order of operations: **B**rackets, **I**ndices, **D**ivision and **M**ultiplication, then **A**ddition and **S**ubtraction.

CALCULATION	RESULT	REASON
`4 + 3 * 5`	`19`	multiplication happens before addition
`(4 + 3) * 5`	`35`	brackets force the addition first
`20 - 6 / 3`	`18.0`	division first, then subtraction
`2 ** 3 + 1`	`9`	the index is evaluated first
`(8 + 4) / (5 - 2)`	`4.0`	both brackets first, then divide

Use brackets whenever they make your intention clearer, even where BIDMAS would already give the right answer. Code is read far more often than it is written.

✧ DID YOU KNOW?

The `%` operator is how a program knows whether a number is even. If `n % 2` gives `0`, the number divides exactly by two. Programmers use the same trick to work out whether a year is a leap year, and to make a clock roll over from 23:59 to 00:00.

PRACTISE 7

1. Work out each of these on paper, then check in the shell: `7 + 2 * 6`, `(7 + 2) * 6`, `25 // 4`, `25 % 4`, `3 ** 4`.
2. A box holds 12 pastéis. Write a program that asks how many pastéis were baked and prints how many full boxes can be filled and how many are left over.
3. Write a program that asks for a number of minutes and prints the equivalent in hours and minutes. Test it with 195 minutes: it should print 3 hours and 15 minutes.
4. Explain why `10 / 2` prints `5.0` while `10 // 2` prints `5`.

UNIT 7.1 · TOPIC 8

Algorithms and flowcharts

An **algorithm** is a precise sequence of steps that solves a problem. Before you type a single line of Python, it pays to plan the algorithm. A **flowchart** is a picture of that plan, and it uses a small set of standard symbols so that any programmer can read it.

SYMBOL	NAME	USED FOR
Rounded box	Terminator	the START and the STOP of the algorithm
Parallelogram	Input or output	reading a value in, or displaying one
Rectangle	Process	a calculation or an assignment
Diamond	Decision	a question with two possible answers
Arrow	Flow line	the order in which steps happen

A flowchart for the minibus problem

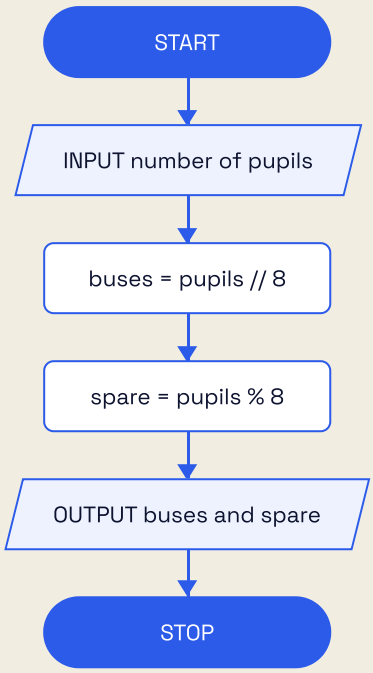


Figure 1.1 A sequence algorithm: every step runs once, in order.

Each step follows the one before it. This arrangement is called a **sequence**, and it is the simplest of the three program structures.

KEYWORDS

algorithm a precise, ordered set of steps that solves a problem

flowchart a diagram that shows an algorithm using standard symbols

sequence program steps carried out one after another in a fixed order

PRACTISE 8

1. Draw a flowchart for an algorithm that asks for the length and width of a classroom and outputs its area.
2. Draw a flowchart for making a cup of tea. Use at least six symbols and include one decision.
3. Explain why a diamond must always have two arrows leaving it.

Selection: making decisions

Selection means choosing between two or more paths. In a flowchart it is the diamond; in Python it is the `if` statement.

The comparison operators

OPERATOR	MEANING	TRUE EXAMPLE
`==`	is equal to	`7 == 7`
`≠`	is not equal to	`7 ≠ 4`
`<`	is less than	`4 < 7`
`>`	is greater than	`7 > 4`
`≤`	is less than or equal to	`7 ≤ 7`
`≥`	is greater than or equal to	`7 ≥ 4`

Take great care with `=` and `==`. A single `=` stores a value; a double `==` asks a question.

Writing an if statement

```
mark = int(input("Enter the test mark: "))
if mark ≥ 50:
    print("Pass")
else:
    print("Needs another attempt")
```

```
Enter the test mark: 63
Pass
```

Two details matter enormously.

- The line ending in `:` opens a block.
- Everything belonging to that block is **indented** by four spaces. Indentation is not

decoration in Python: it is how the language knows which lines belong together.

Choosing between more than two paths

`elif`, short for "else if", adds further questions. Python checks each in turn and runs the first one that is true.

```
temp = float(input("Water temperature in C: "))
if temp ⩾ 25:
    print("Warm enough for a long swim")
elif temp ⩾ 18:
    print("Bracing but pleasant")
else:
    print("The Atlantic wins today")
```

```
Water temperature in C: 19.5
Bracing but pleasant
```

The same decision as a flowchart

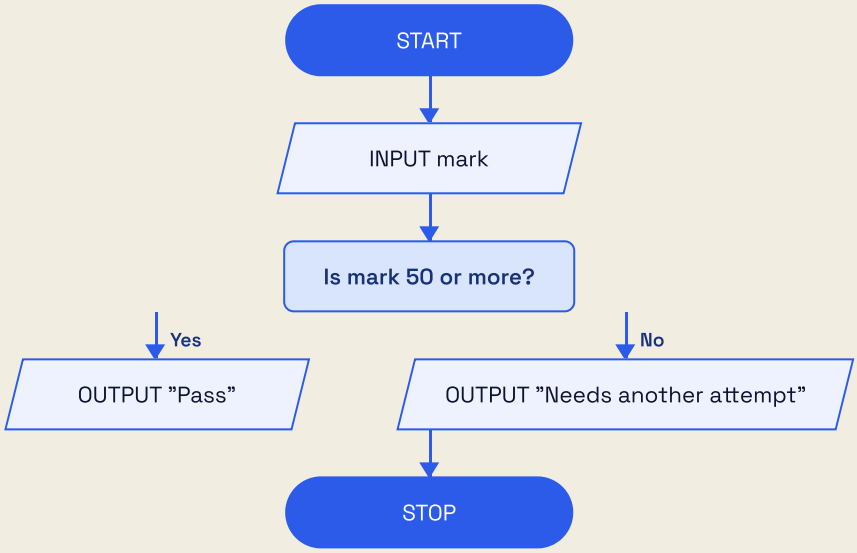


Figure 1.2 A selection algorithm: the diamond sends the flow down one of two branches.

KEYWORDS

selection choosing between different paths through a program depending on a condition

condition an expression that is either True or False

indentation the spaces at the start of a line that show which block a statement belongs to

- PRACTISE 9

1. Write a program that asks for a pupil's age and prints `Lower Secondary` if the age is between 11 and 14 inclusive, and `Check the year group` otherwise.
2. Write a program that asks for a number and reports whether it is even or odd. Use the `%` operator.
3. Extend the swimming program with a fourth message for temperatures below 12 degrees.
4. Find the mistake: `if mark = 50:` Explain what is wrong and correct it.

UNIT 7.1 · TOPIC 10

Errors, testing and debugging

Every programmer writes bugs. What separates a good programmer from a frustrated one is a calm, systematic way of finding them.

Three families of error

TYPE	WHAT HAPPENS	EXAMPLE	HOW YOU SPOT IT
Syntax error	Python cannot understand the line, so nothing runs	`print("Hello"`	an error message appears before the program starts
Runtime error	the program starts, then stops part way	`int("seven")`	a `Traceback` appears mid-run
Logic error	the program runs happily and gives the wrong answer	using `+` where you meant `*`	only testing reveals it

Logic errors are the most dangerous, because the computer never complains.

Reading a traceback

```
Traceback (most recent call last):
  File "fare.py", line 3, in <module>
    total = passengers * fare
NameError: name 'passenger' is not defined
```

Read a traceback from the bottom upwards. The last line names the problem, and the line above tells you where to look. Here the variable was created as `passengers` but used as `passenger`.

Building a test plan

A **test plan** decides, before you run anything, what you will type in and what should come out. Good plans include three kinds of data.

- **Normal data**: sensible values the program will usually meet.

- **Boundary data**: values right at the edge of what is allowed.
- **Erroneous data**: values that should be rejected.

Here is a test plan for the pass mark program from Topic 9, where the pass mark is 50.

TEST	TYPE OF DATA	INPUT	EXPECTED OUTPUT	REASON
1	Normal	`63`	`Pass`	comfortably above the pass mark
2	Normal	`31`	`Needs another attempt`	comfortably below
3	Boundary	`50`	`Pass`	50 must count as a pass
4	Boundary	`49`	`Needs another attempt`	one below the pass mark
5	Erroneous	`fifty`	an error, or a polite message	not a whole number

Test 3 is the one that catches the classic mistake of writing `>` when you meant `>=`.

A debugging routine that works

1. Read the error message properly, from the bottom up.
2. Go to the line it names, and look at the line above it too.
3. Check spelling, capital letters, brackets, colons and quotation marks.
4. Check the indentation of every block.
5. Add temporary `print()` lines to see what your variables actually contain.
6. Change one thing at a time, then test again.

KEYWORDS

syntax error a mistake in the rules of the language that stops the program running

runtime error an error that appears while the program is running

logic error a fault that lets the program run but produces the wrong result

test plan a prepared table of inputs and expected outputs used to check a program

debugging finding and correcting errors in a program

PRACTISE 10

1. Classify each fault as syntax, runtime or logic: a missing closing bracket; dividing by a variable that holds zero; calculating an average by dividing by 3 when there are 4 values.
2. Write a test plan with five rows for a program that decides whether a parcel weighing up to 20 kg can be posted.
3. The program below should print the area of a rectangle but prints something else. Find and fix the bug.

```
length = int(input("Length: "))
width = int(input("Width: "))
area = length + width
print("Area:", area)
```

→ GO FURTHER: SUB-ROUTINES

As programs grow, repeating the same lines becomes tiresome and error-prone. A **sub-routine** is a named piece of code you write once and call whenever you need it. In Python you create one with `def`.

```
def show_fare(people, price):
    total = people * price
    print("Fare for", people, "people:", total)

show_fare(2, 3.20)
show_fare(5, 3.20)
```

```
Fare for 2 people: 6.4
Fare for 5 people: 16.0
```

The sub-routine is defined once and used twice. If the fare rules change, you edit one place. In a flowchart a sub-routine is drawn as a rectangle with a double line at each side.

▲ CHALLENGE YOURSELF

Build a **ferry ticket machine** for the crossing from Cais do Sodré to Cacilhas.

The rules are these. A single adult ticket costs €1.55. Passengers aged 12 or under, and those aged 65 or over, pay half price. A group of 10 or more passengers receives a further 20 per cent off the whole order, applied after any age reductions.

Your program should ask for the number of passengers, ask for the age of each one, then print the total to the nearest cent.

Prove it works with this test plan.

TEST	PASSENGERS	AGES	EXPECTED TOTAL	WORKING
1	2	30, 41	€3.10	2 × 1.55
2	3	30, 10, 70	€3.10	1.55 + 0.775 + 0.775
3	10	ten adults	€12.40	15.50 less 20 per cent

★ FINAL PROJECT

The Prime School trip calculator

The Geography department is taking Year 7 to the Serra da Arrábida. They need a Python program that works out what the trip will cost each pupil, and they need to be certain the figures are right. That program is your job.

THE BRIEF

Your program must:

- ask how many pupils are going
- ask the coach hire cost for the day
- ask the entry fee for one pupil
- divide the coach cost evenly between the pupils
- add the entry fee to give the cost per pupil
- print the cost per pupil and the total cost of the trip
- print a warning if the cost per pupil goes above €15.00

MILESTONES

1. **Plan the algorithm**
   Draw a flowchart using the standard symbols. Include the decision that produces the warning.
2. **Write a test plan**
   Prepare at least six rows covering normal, boundary and erroneous data. Work out every expected answer by hand first.
3. **Build the program**
   Use sensible variable names, cast every input, and comment the tricky lines.
4. **Test and debug**
   Run every row of your plan. Record what actually happened next to what you expected.
5. **Evaluate**
   Write a short paragraph: what works, what you would improve, and what you found hardest.

WORKED EXAMPLE TO CHECK AGAINST

```
Pupils: 25
Coach hire: 250
Entry fee: 4.50

Cost per pupil: 14.50
Total cost: 362.50
```

Coach: 250 ÷ 25 = 10.00 per pupil. Plus 4.50 entry = 14.50 each. Total 25 × 14.50 = 362.50. No warning, because 14.50 is below 15.00.

HOW YOUR WORK WILL BE JUDGED

STRAND	SECURE
Algorithm	flowchart uses correct symbols and shows the decision
Program	runs without errors and gives correct figures
Data types	inputs cast correctly; money handled as a real number
Testing	six or more rows including boundary values at €15.00
Clarity	helpful names, useful comments, tidy output
Evaluation	honest, specific and suggests real improvements

UNIT 7.1

Evaluation and review

Think carefully about your work in this unit. For each statement, colour the circle that matches how confident you feel. Green means you could teach it to somebody else, amber means you can do it with your notes open, and red means you would like to go over it again.

I CAN ...	GREEN	AMBER	RED
explain the difference between block-based and text-based programming	○	○	○
use the IDLE shell and script mode for the right jobs	○	○	○
create variables with clear names and change their values	○	○	○
collect input and display well-presented output	○	○	○
name the four data types and choose the right one	○	○	○
cast values between types, and explain why input needs casting	○	○	○
use all seven arithmetic operators, including `//` and `%`	○	○	○
apply BIDMAS and use brackets to make my intention clear	○	○	○
draw a flowchart with the correct standard symbols	○	○	○
write selection statements using the comparison operators	○	○	○
tell syntax, runtime and logic errors apart	○	○	○
write a test plan with normal, boundary and erroneous data	○	○	○

✓ WHAT CAN YOU DO?

- ☐ Move confidently from Scratch blocks to typed Python
- ☐ Save, run and reopen a program in script mode
- ☐ Store values in well-named variables
- ☐ Ask the user questions and cast their answers
- ☐ Work with strings, integers, reals and Booleans
- ☐ Calculate using `+ - * / // % **`
- ☐ Predict results using BIDMAS
- ☐ Plan an algorithm as a flowchart
- ☐ Branch a program with `if`, `elif` and `else`
- ☐ Build and follow a test plan
- ☐ Read a traceback and debug calmly
- ☐ Write a sub-routine with `def`

UNIT

7.2

Decomposing problems

Creating a smart solution

✓ LEARNING OUTCOMES

In this unit you will learn to:

- ✓ explain the difference between systems software and application software
- ✓ break a large problem into smaller parts through decomposition
- ✓ write and transfer Python programs for the micro:bit
- ✓ use the display, buttons and sensors of a physical device
- ✓ read values from the temperature, light and accelerometer sensors
- ✓ use a flowchart to locate a fault in an algorithm
- ✓ plan an algorithm with a flowchart before writing any code
- ✓ choose sensible data types for sensor readings
- ✓ build a test plan with normal, boundary and erroneous data
- ✓ evaluate a finished program honestly against its original brief

◆ GET STARTED

A greenhouse in the Alentejo waters itself at four in the morning. Nobody is awake. How does it decide?

Think about a device that acts without being told each time. Discuss with a partner.

- What does the device need to measure before it can decide anything?
- What does it do with that measurement?
- What should happen if the measurement looks impossible, perhaps a temperature of 300 degrees?

Any device like this follows the same three-step loop: **sense**, **decide**, **act**. A sensor turns something physical into a number. A program compares that number against a rule. An output does something about it.

In this unit you will build such devices yourself using the BBC micro:bit. Before that, you will learn the single most useful skill in computing: taking a problem that feels too big and cutting it into parts small enough to solve.

▮ KEYWORDS

decomposition breaking a large problem into smaller problems that can each be solved separately

sensor a component that measures something physical and reports it as a number

embedded system a computer built into a device to control it, rather than a general-purpose computer

✦ WARM UP

Beatriz wants a device that reminds her to drink water during the school day.

1. Write down three things the device must be able to measure or keep track of.
2. Write down two things it could do to get her attention.
3. Break the whole problem into four smaller problems, each of which could be solved on its own.
4. Which of your four parts do you think is hardest? Say why.

↺ DO YOU REMEMBER?

Unit 7.1 gave you everything you need to start.

- A **variable** stores a value: `temp = 21`
- **Casting** converts between types: `int("21")`
- **Selection** chooses a path: `if temp > 25:`
- **Indentation** shows which lines belong to a block.
- A **flowchart** plans the algorithm before you type it.

The only new idea in this unit is that the numbers now come from the physical world instead of from a keyboard.

UNIT 7.2 · TOPIC 1

All about software

Software falls into two families, and the difference explains what happens when things go wrong.

Systems software runs the machine. **Application software** does a job for you.

	SYSTEMS SOFTWARE	APPLICATION SOFTWARE
Purpose	operates and maintains the computer	completes a task for the user
Started by	the machine, automatically	the user, deliberately
Examples	operating system, drivers, utilities	browser, spreadsheet, IDLE, games
If it fails	the whole machine may stop	usually just that program stops

Software on a device with no screen

The micro:bit has no operating system in the ordinary sense. It runs a small piece of systems software called **firmware**, which starts the board and then hands control to your program. This arrangement is called an **embedded system**: a computer built into a device to control it.

Embedded systems are everywhere. A washing machine, a car's braking system, a traffic light and a heart monitor all contain one. They usually do a single job, run the same program for years, and must never crash.

✧ DID YOU KNOW?

There are far more embedded computers in the world than laptops and phones combined. A modern car contains somewhere between fifty and a hundred separate processors, controlling everything from the engine timing to the windows. Most of them run software that will never be updated once the car leaves the factory.

• PRACTISE 1

1. Sort these into systems or application software: a printer driver, a photo editor, firmware, Android, a spreadsheet, a disc backup tool.
2. Give three examples of embedded systems in your own home, and say what each one senses.
3. Explain why an embedded system in a lift must be more reliable than a game on a phone.

UNIT 7.2 · TOPIC 2

Decomposition: cutting a problem down

Decomposition means breaking a problem into smaller problems. It is the most useful habit in computing, because a small problem can be solved, tested and understood, while a large one cannot.

Decomposing a smart bicycle light

Gonçalo wants a light that switches on by itself when it gets dark and flashes when he brakes. Written like that, it is one intimidating problem. Decomposed, it becomes five easy ones.

PART	THE SMALLER PROBLEM	SOLVED BY
1	Measure how bright it is	read the light sensor
2	Decide whether that counts as dark	compare against a threshold
3	Turn the light on or off	write to the display
4	Detect braking	read the accelerometer
5	Flash while braking	a loop with pauses

Each row can be built and tested on its own. When all five work, joining them is straightforward.

The three questions of decomposition

1. **What are the inputs?** What must the system measure or be told?
2. **What are the outputs?** What must it produce or change?

3. **What steps connect them?** What has to happen in between, in order?

KEYWORDS

decomposition breaking a problem into smaller sub-problems

threshold a value used as the dividing line in a decision

firmware systems software stored permanently in a device to start and control it

---

PRACTISE 2

1. Decompose the problem of a device that counts how many people enter the school library. Give at least four parts.
2. For each part, say whether it is an input, an output or a step in between.
3. Decompose a device that warns when a classroom is too noisy. Which sensor would you need?

UNIT 7.2 · TOPIC 3

The micro:bit and its Python environment

The BBC micro:bit is a small board with a 5 by 5 grid of red LEDs, two buttons, and sensors for temperature, light and movement, all built in.

DISPLAY

25 LEDs in a 5 × 5 grid, each with 10 brightness levels

BUTTONS

Button A and button B, plus a reset on the back

SENSORS

Temperature, light, accelerometer, compass

Writing and transferring a program

You write micro:bit Python in a browser editor, then send the finished program to the board.

1. Open the editor and start a new project.
2. Type your program.
3. Connect the micro:bit with a USB cable.
4. Click **Send to micro:bit**, or **Download** and drag the file onto the MICROBIT drive.
5. The yellow light on the back flashes while it transfers, then the board restarts and runs.

Every micro:bit program begins with the same line, which brings in all the commands for the display, buttons and sensors.

```
from microbit import *

display.scroll("Ola!")
```

The board scrolls the message across the LED grid once, then stops.

The display

COMMAND	WHAT IT DOES
`display.scroll("text")`	slides text across the grid
`display.show(Image.HEART)`	shows a built-in picture
`display.show(7)`	shows a single character or digit
`display.clear()`	turns every LED off
`display.set_pixel(x, y, b)`	sets one LED, brightness `b` from 0 to 9

The grid is numbered from 0, with `x` running left to right and `y` running top to bottom. The centre LED is therefore `(2, 2)`, not `(3, 3)`.

```
from microbit import *

display.clear()
display.set_pixel(0, 0, 9)
display.set_pixel(2, 2, 5)
display.set_pixel(4, 4, 9)
```

This lights the top left corner brightly, the centre at half brightness, and the bottom right corner brightly.

Waiting, and repeating for ever

`sleep(ms)` pauses the program for a number of milliseconds. `while True:` repeats for ever, which is exactly what an embedded device usually needs to do.

```
from microbit import *

while True:
    display.show(Image.HEART)
    sleep(500)
    display.clear()
    sleep(500)
```

The heart flashes on for half a second, off for half a second, until the board is unplugged.

KEYWORDS

LED a small light, twenty-five of which form the micro:bit display

millisecond one thousandth of a second, the unit used by `sleep()`

infinite loop a loop such as `while True:` that repeats until the device is switched off

PRACTISE 3

1. Write a program that scrolls your name, then shows a heart for two seconds.
2. Write a program that lights only the four corners of the display.
3. Write a program that flashes an image on for 200 ms and off for 800 ms, for ever.
4. Explain why the centre pixel is `(2, 2)` rather than `(3, 3)`.

UNIT 7.2 · TOPIC 4

Buttons: responding to a user

A button gives your program its first real input from the outside world.

COMMAND	WHAT IT REPORTS
`button_a.is_pressed()`	`True` while button A is held down
`button_b.is_pressed()`	`True` while button B is held down
`button_a.was_pressed()`	`True` once if A has been pressed since you last asked

`is_pressed()` asks about right now. `was_pressed()` remembers a press that has already finished, which is far more useful for counting things.

A tally counter for the library door

```
from microbit import *

count = 0
while True:
    if button_a.was_pressed():
        count = count + 1
        display.show(count)
    if button_b.was_pressed():
        count = 0
        display.show(Image.NO)
        sleep(500)
        display.clear()
```

Button A adds one to the count and shows it. Button B resets the count to zero, shows a cross for half a second, then clears the display.

Notice that `count` is created **before** the loop. Putting it inside would reset it to zero on every pass, and the counter would never climb above one.

→ GO FURTHER: NUMBERS ABOVE NINE

`display.show(count)` can only show one character at a time, so a count of 12 appears as a `1` followed by a `2`, which is easy to misread. Use `display.scroll(count)` once the count passes nine, and the number slides across clearly.

```
if count > 9:
    display.scroll(count)
else:
    display.show(count)
```

• PRACTISE 4

1. Write a program that shows `A` when button A is held and `B` when button B is held.
2. Modify the tally counter so it counts down from 10 instead of up from 0.
3. Explain what would go wrong if `count = 0` were written inside the `while True:` loop.
4. Write a program where pressing A and B together clears the display.

UNIT 7.2 · TOPIC 5

Using sensors

A **sensor** turns something physical into a number your program can compare.

SENSOR	COMMAND	RETURNS
Temperature	`temperature()`	degrees Celsius, a whole number
Light	`display.read_light_level()`	0 (dark) to 255 (bright)
Movement	`accelerometer.get_x()`	roughly −1024 to 1024
Gesture	`accelerometer.was_gesture("shake")`	`True` if shaken

A classroom thermometer

```
from microbit import *

while True:
    temp = temperature()
    if temp ⩾ 26:
        display.show(Image.ANGRY)
    elif temp ⩽ 17:
        display.show(Image.ASLEEP)
    else:
        display.show(Image.HAPPY)
    sleep(2000)
```

The board checks the temperature every two seconds and shows a face for hot, cold or comfortable.

Choosing a threshold honestly

A threshold should come from measurement, not from a guess. To set the dark threshold for Gonçalo's bicycle light, record real readings first.

SITUATION	LIGHT READING	CONCLUSION
Bright sunshine outdoors	245	clearly daylight
Overcast afternoon	160	still daylight
Classroom with lights on	95	indoors, lamp needed
Dusk outdoors	40	light should switch on
Covered with a hand	2	dark

A threshold of 50 sits comfortably between dusk and the lit classroom, so the light comes on at dusk without flickering on indoors.

Data types for sensor readings

READING	SENSIBLE TYPE	WHY
Temperature in whole degrees	integer	the sensor reports whole numbers
A calculated average temperature	real	dividing produces a decimal
Light level	integer	always a whole number from 0 to 255
Is it dark?	Boolean	the answer is yes or no

✧ DID YOU KNOW?

The micro:bit has no separate light sensor. It measures light using the LEDs of the display itself, run briefly in reverse. An LED that normally emits light will generate a tiny current when light falls on it, and the board measures that current. This is why the reading changes if you cover the display with your thumb.

● PRACTISE 5

1. Write a program that scrolls the current temperature every three seconds.
2. Record five light readings in different places around your school and build a table like the one above.
3. Using your own readings, choose a threshold for a corridor light and justify it in one sentence.
4. Write a program that shows a different image when the board is shaken.

UNIT 7.2 · TOPIC 6

Planning with flowcharts

A flowchart shows the algorithm before you type it. Planning first is quicker overall, because mistakes are far cheaper to fix in a diagram than in code.

The sense, decide, act loop

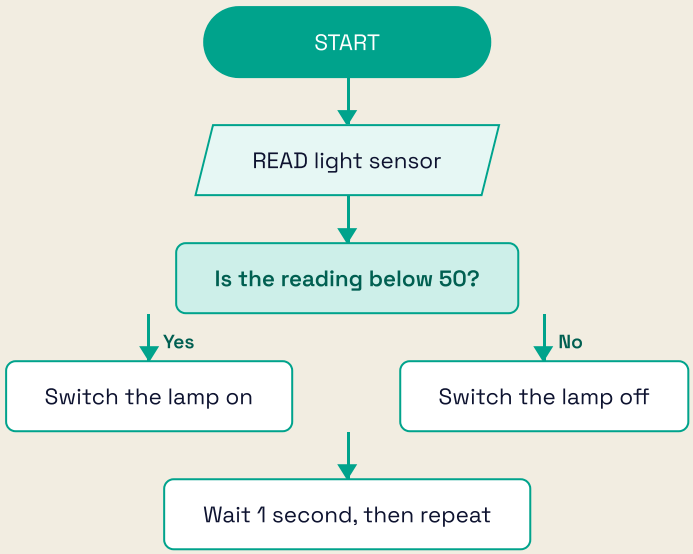


Figure 2.1 The sense, decide, act loop that sits behind almost every smart device.

Turning that diagram into code is now mechanical.

```
from microbit import *

while True:
    level = display.read_light_level()
    if level < 50:
        display.show(Image.SQUARE)
    else:
        display.clear()
    sleep(1000)
```

Finding a fault with a flowchart

A flowchart is also a debugging tool. When a program misbehaves, draw what it actually does and compare it against what it should do. The difference is almost always visible.

Inês wrote a program to warn when a greenhouse is too cold. It never warns, even at 5 degrees.

```
from microbit import *

while True:
    temp = temperature()
    if temp > 10:
        display.show(Image.SAD)
    sleep(1000)
```

Tracing the diamond makes the fault obvious. Her condition asks whether the temperature is **above** 10, so the warning appears when the greenhouse is warm and stays hidden when it is cold. The comparison operator is the wrong way round, and it should be `temp < 10`.

This is a **logic error**: the program runs perfectly and produces the wrong answer.

• PRACTISE 6

1. Draw a flowchart for a device that shows a tick when the temperature is between 18 and 24 degrees inclusive, and a cross otherwise.
2. Convert your flowchart into Python and test it.
3. A program should flash a warning when a light reading rises above 200 but flashes constantly in a dark room. Suggest the likely fault.
4. Explain why drawing the flowchart first usually saves time.

UNIT 7.2 · TOPIC 7

Testing a physical device

Testing a device that senses the real world needs more care than testing a calculator, because you cannot always type the value you want to test.

Normal, boundary and erroneous data

Here is a test plan for the greenhouse warning, where the alert should appear below 10 degrees.

TEST	TYPE	INPUT	EXPECTED	HOW TO PRODUCE IT
1	Normal	21 °C	no warning	ordinary room
2	Normal	4 °C	warning	put the board in a fridge
3	Boundary	10 °C	no warning	10 is not below 10
4	Boundary	9 °C	warning	one degree below
5	Erroneous	sensor disconnected	no crash	unplug and observe

Rows 3 and 4 matter most. They are the only tests that catch the difference between `<` and `⩽`, which is the single most common mistake in this kind of program.

Testing without a fridge

You cannot always reach a boundary value in the real world. The professional answer is to test the **decision** separately from the **sensor**, by writing the rule as a function and calling it with any value you like.

```
def is_too_cold(temp):
    return temp < 10

print(is_too_cold(21))
print(is_too_cold(10))
print(is_too_cold(9))
print(is_too_cold(4))
```

```
False
False
True
True
```

Every boundary is now testable in a second, with no fridge required. Once the function is proved correct, the device simply calls it with the real reading.

KEYWORDS

normal data typical values the program will usually meet

boundary data values at the very edge of a decision, where behaviour changes

erroneous data values that should be rejected or handled safely

logic error a fault that lets the program run but produces the wrong result

PRACTISE 7

1. Write a test plan of six rows for a device that warns when a room is louder than a set level.
2. Write `is_too_bright(level)` returning `True` above 200, and test the boundaries 199, 200 and 201.
3. Explain why the boundary rows of a test plan are more valuable than the normal rows.
4. Suggest how you would test a device meant to detect a fall, without dropping it.

Evaluating a program

Evaluation means judging honestly whether the finished thing solves the original problem. It is not a summary of what you did, and it is not an apology.

What a good evaluation contains

- **Against the brief.** Take each requirement in turn and say whether it is met, with evidence.
- **Evidence from testing.** Refer to specific rows of your test plan and their real results.
- **Weaknesses.** What does it still do badly, and in what situation?
- **Improvements.** What would you change, and what difference would it make?

Two evaluations of the same project

> "I made the bike light and it works well. I like it. If I had more time I would make it better."

That says nothing checkable. Compare it with this.

> "The light meets three of the four requirements. It switches on below a reading of 50, proved > by tests 3 and 4, and it flashes when braking, proved by test 6. It does not meet the > requirement to stay off in a lit garage, because a garage reads 55 and the threshold is too > close. I would raise the threshold to 35 and re-run tests 3 to 5. The battery lasts about six > hours, which is short for a winter week, so I would also increase the sleep between readings > from 100 ms to 500 ms."

The second is specific, references evidence, admits a failure and proposes a measurable fix.

→ GO FURTHER: MEASURE BEFORE YOU IMPROVE

Any claim in an evaluation should be measurable. "It is slow" is an opinion; "it takes 1.4 seconds to react, and it should react within 0.5 seconds" is a fact that tells you what to fix and lets you prove you fixed it.

• PRACTISE 8

1. Rewrite this evaluation so it is specific: "My counter is quite good but sometimes it goes wrong."
2. List four requirements for a device that reminds you to drink water, then write an honest evaluation sentence for each.
3. Explain the difference between testing and evaluating.

▲ CHALLENGE YOURSELF

Build a **frost alarm for the Douro vineyards**.

Vines are damaged when the temperature drops to 2 °C or below. Growers need warning before the temperature actually reaches freezing.

Your device must:

- read the temperature every 5 seconds
- show a happy face **above** 5 °C
- show a warning image **from above 2 °C up to and including 5 °C**
- flash a cross continuously at **2 °C or below**
- keep a count of how many readings have been at or below 2 °C, shown when button A is pressed

Notice that the three bands must cover every possible temperature with no gaps and no overlaps. Check the edges carefully: a reading of 2.9 °C has to land in exactly one band.

Write the decision as a function `frost_state(temp)` returning the text `"safe"`, `"watch"` or `"alarm"`, then prove it with this plan.

TEST	TEMPERATURE	EXPECTED	WHY
1	12	safe	comfortably warm
2	5.1	safe	just above the watch band
3	5	watch	exactly on the upper edge
4	3	watch	inside the band
5	2.9	watch	just above the alarm edge
6	2	alarm	exactly on the alarm edge
7	−4	alarm	hard frost

★ FINAL PROJECT

A smart solution for Prime School

Choose one real problem in your school and solve it with a micro:bit. The device must sense something, decide something, and act on that decision. You will be judged as much on your planning and testing as on the finished program.

CHOOSE ONE

- a corridor light that switches on only when it is genuinely dark
- a greenhouse monitor for the school garden
- a noise meter for the library
- a step counter for the sports department
- a queue counter for the canteen door

MILESTONES

1. **Decompose**
   Split your problem into at least four parts. State the inputs, the outputs and the steps between.
2. **Plan the algorithm**
   Draw a flowchart with correct symbols, including every decision.
3. **Gather real readings**
   Record at least five sensor readings in different conditions, and use them to choose your threshold. Justify the number you picked.
4. **Write the decision as a function**
   So you can test every boundary without a fridge or a dark room.
5. **Build and test**
   Six or more rows including boundaries. Record what actually happened beside what you expected.
6. **Evaluate**
   Each requirement met or not, with evidence, one honest weakness and one measurable improvement.

SKELETON TO BUILD ON

```
from microbit import *

# The decision, kept separate so
# it can be tested on its own.
def state(reading):
    if reading < 50:
        return "on"
    return "off"

while True:
    level =
display.read_light_level()
    if state(level) == "on":
        display.show(Image.SQUARE)
    else:
        display.clear()
    sleep(1000)
```

HOW YOUR WORK WILL BE JUDGED

STRAND	SECURE
Decomposition	four or more sensible parts, inputs and outputs identified
Algorithm	flowchart uses correct symbols and matches the code
Threshold	chosen from real recorded readings, and justified
Program	runs on the board and behaves as planned
Testing	six or more rows, boundaries included, real results recorded
Evaluation	specific, evidenced, admits a weakness, proposes a measurable fix

UNIT 7.2

Evaluation and review

Think carefully about your work in this unit. For each statement, colour the circle that matches how confident you feel. Green means you could teach it to somebody else, amber means you can do it with your notes open, and red means you would like to go over it again.

I CAN ...	GREEN	AMBER	RED
explain the difference between systems and application software	○	○	○
describe what an embedded system is and give examples	○	○	○
decompose a large problem into smaller solvable parts	○	○	○
write a micro:bit program and transfer it to the board	○	○	○
control the display, including individual pixels	○	○	○
respond to button presses, and know when to use was_pressed	○	○	○
read the temperature, light and accelerometer sensors	○	○	○
choose a threshold from real recorded readings	○	○	○
select sensible data types for sensor values	○	○	○
plan an algorithm as a flowchart before coding	○	○	○
use a flowchart to locate a logic error	○	○	○
build a test plan with normal, boundary and erroneous data	○	○	○
write an honest, specific and evidenced evaluation	○	○	○

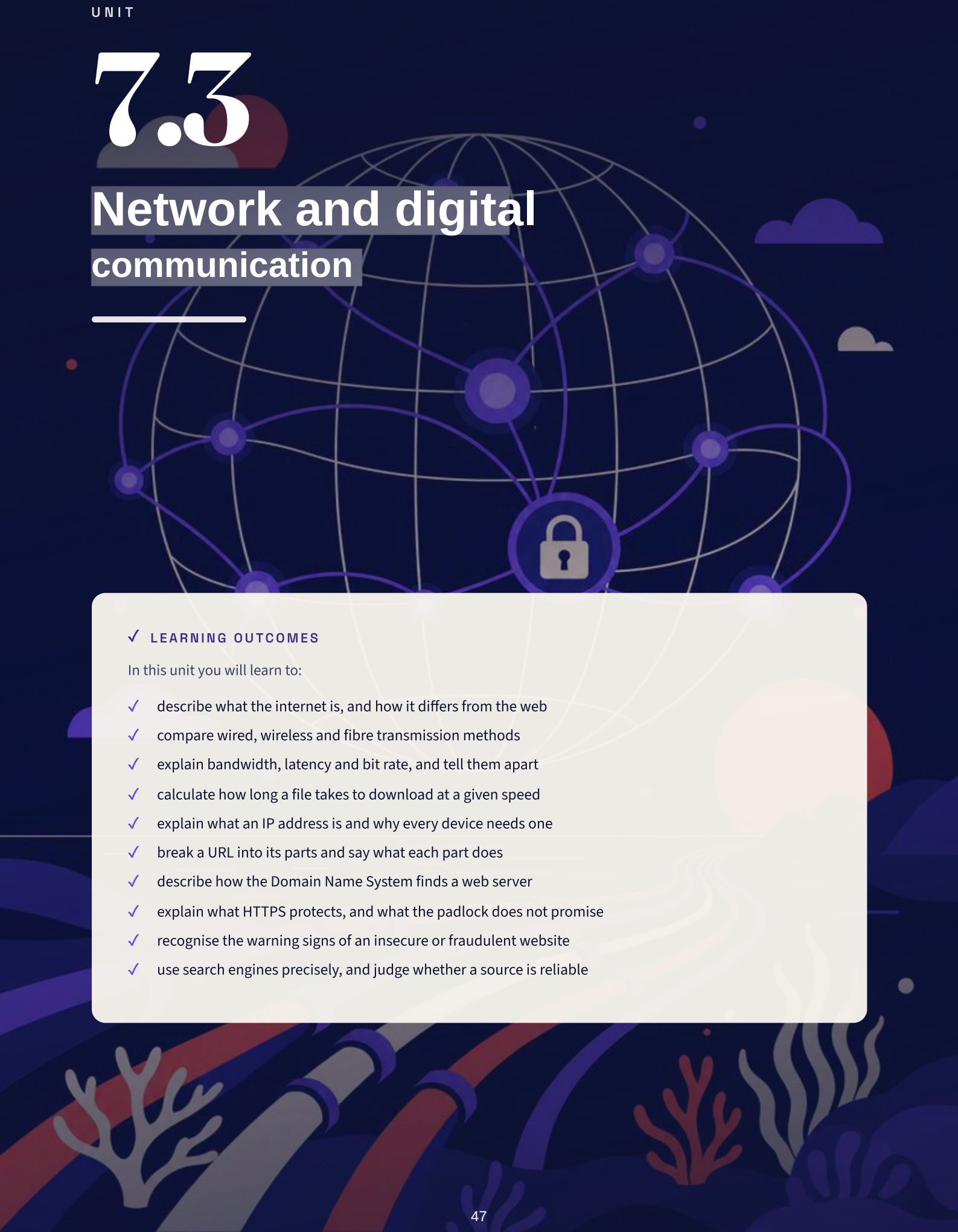
✓ WHAT CAN YOU DO?

- ☐ Classify software as systems or application
- ☐ Recognise embedded systems around you
- ☐ Break a big problem into small ones
- ☐ Send a Python program to a micro:bit
- ☐ Scroll text, show images and set single pixels
- ☐ Read and respond to both buttons
- ☐ Take readings from three different sensors
- ☐ Justify a threshold with recorded evidence
- ☐ Plan with a flowchart, then code from it
- ☐ Separate the decision from the sensor to test it
- ☐ Test boundaries deliberately
- ☐ Evaluate your own work honestly

UNIT

7.3

Network and digital communication



✓ LEARNING OUTCOMES

In this unit you will learn to:

- ✓ describe what the internet is, and how it differs from the web
- ✓ compare wired, wireless and fibre transmission methods
- ✓ explain bandwidth, latency and bit rate, and tell them apart
- ✓ calculate how long a file takes to download at a given speed
- ✓ explain what an IP address is and why every device needs one
- ✓ break a URL into its parts and say what each part does
- ✓ describe how the Domain Name System finds a web server
- ✓ explain what HTTPS protects, and what the padlock does not promise
- ✓ recognise the warning signs of an insecure or fraudulent website
- ✓ use search engines precisely, and judge whether a source is reliable

GET STARTED

You type six letters and press Enter. Somewhere, a machine you will never see answers you in less time than a blink.

Think about what happens when you load a page. Discuss with a partner.

- Where is the page actually stored before it appears on your screen?
- How does your laptop know where to look?
- What physically carries the page to you: wires, radio, light, or all three?

The honest answer is that a request leaves your device, crosses a chain of machines that may include a fibre cable on the floor of the Atlantic, finds one specific computer among billions, and comes back. All of it happens in well under a second.

In this unit you will follow that journey step by step. You will also learn how to travel it safely, because the same network that delivers a library delivers people who would like your password.

KEYWORDS

internet the global network of connected networks that carries data between devices

World Wide Web the collection of linked pages and files that travels over the internet

server a computer that stores information and sends it out when another computer asks

WARM UP

Put these in the order they happen when Beatriz loads a web page. One is a distractor and does not belong at all.

1. The browser displays the page.
2. The server sends the page back.
3. Beatriz types the address and presses Enter.
4. The printer warms up.
5. The browser asks the DNS for the server's address.
6. The browser sends a request to that address.

Now answer this: which step would fail first if the school's internet connection were unplugged?

↺ DO YOU REMEMBER?

You already know more than you expect.

- From Unit 7.1: data has **types**, and everything is ultimately numbers.
- From Unit 7.2: a **sensor** turns something physical into a number a program can use.
- A **bit** is a single 1 or 0. Eight bits make a **byte**.

This unit adds one idea: those bits can travel, and the journey has rules.

UNIT 7.3 · TOPIC 1

Getting online

The **internet** is a network of networks. No single company owns it. It is millions of separate networks that have agreed to pass each other's data using shared rules called **protocols**.

The **World Wide Web** is not the same thing. The web is one service that runs on the internet, made of pages and links. Email, video calls and app updates also use the internet but are not part of the web. The internet is the road; the web is one kind of traffic.

The chain from your desk to the world

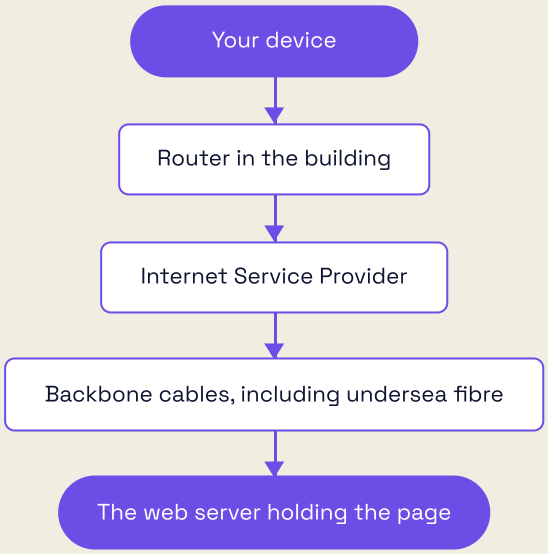


Figure 3.1 Every request travels this chain, and the reply travels back along it.

- A **router** joins your local network to the wider internet and directs traffic to the right device.
- An **Internet Service Provider**, or ISP, is the company that connects your building to the rest of the internet.

- The **backbone** is the set of very high capacity cables linking cities and continents.

Data travels in packets

Large files are not sent in one piece. They are chopped into small **packets**, each carrying the destination address and its position in the sequence. Packets may take different routes and arrive out of order, and the receiving device puts them back in the right order.

This sounds fussy but it is why the internet is so robust. If one route fails, packets simply go another way, and nothing needs to start again from the beginning.

✧ DID YOU KNOW?

About 99 per cent of international internet traffic travels through undersea fibre optic cables, not satellites. Portugal is a major landing point: cables from Brazil, west Africa and North America come ashore near Lisbon and in the Algarve. Some of those cables are barely thicker than a garden hose, and they carry the traffic of entire continents.

▮ KEYWORDS

protocol an agreed set of rules that lets different systems communicate

router a device that directs data between networks and to the correct device

ISP Internet Service Provider, the company connecting you to the internet

packet a small block of data with an address, part of a larger transmission

● PRACTISE 1

1. Explain in your own words the difference between the internet and the World Wide Web.
2. Name three services that use the internet but are not part of the web.
3. Explain one advantage of splitting a file into packets rather than sending it whole.
4. Draw the chain from your device to a web server, labelling every stage.

UNIT 7.3 · TOPIC 2

Transmission methods

Data travels as electrical signals along copper, as pulses of light along glass, or as radio waves through the air.

METHOD	CARRIES DATA AS	TYPICAL USE	STRENGTHS	WEAKNESSES
Copper cable	electrical signals	office and school wiring	cheap, reliable, no interference from walls	signal fades over distance, slower than fibre
Fibre optic	pulses of light in glass	backbone, undersea, modern homes	very fast, very long distances, immune to electrical interference	costly to install, fragile if bent sharply
Wireless (Wi-Fi)	radio waves	laptops and phones in a building	no cables, easy to move about	walls weaken it, shared between users, easier to intercept
Mobile (4G, 5G)	radio waves to masts	phones out and about	works almost anywhere	depends on signal strength, data may be charged

Why fibre is so much faster

In copper, the signal is a changing voltage that weakens as it travels and picks up interference from nearby electrical equipment. In fibre, the signal is light bouncing along a glass strand thinner than a hair. Light loses very little energy over distance and is completely unaffected by electrical noise, so a fibre can carry far more data, far further, with fewer repeaters.

Choosing sensibly

- A school computer room should be **wired**: dozens of machines in one place, all needing steady speed.
- A tablet used around the building needs **wireless**: mobility matters more than raw speed.
- A cable between Lisbon and Brazil must be **fibre**: nothing else crosses an ocean.

PRACTISE 2

1. Recommend a transmission method for each, with a reason: a school library's fixed desktops; a teacher's tablet; the link between two school buildings 400 m apart; a phone on the 28 tram.
2. Explain why fibre is used for undersea cables rather than copper.
3. Give two reasons a wireless connection might be slower at 15:00 than at 08:00 in a school.
4. Inês says wireless is always more convenient than wired. Give one situation where she is wrong.

UNIT 7.3 · TOPIC 3

Transmission characteristics

Three measurements describe a connection. People confuse them constantly, so learn the difference properly.

BANDWIDTH

How much data can travel per second. Measured in Mbps.

LATENCY

How long one message takes to make the round trip. Measured in ms.

BIT RATE

The actual rate achieved right now, often below the bandwidth.

A useful picture: bandwidth is how many lanes the motorway has, latency is how long the journey takes, and bit rate is how fast the traffic is genuinely moving today.

Bits and bytes: the trap

Connection speeds are quoted in **megabits** per second (Mbps). File sizes are given in **megabytes** (MB). One byte is eight bits, so:

time in seconds = file size in MB × 8 ÷ speed in Mbps

Forgetting the 8 makes every answer eight times too optimistic, which is why downloads always feel slower than advertised.

Worked example

Gonçalo downloads a 600 MB documentary about the Douro on a 50 Mbps connection.

- Convert the file to megabits: 600 × 8 = 4800 Mb
- Divide by the speed: 4800 ÷ 50 = 96 seconds
- That is 1 minute 36 seconds.

On the school's older 10 Mbps line the same file takes 4800 ÷ 10 = 480 seconds, which is 8 minutes exactly.

FILE	SPEED	WORKING	TIME
150 MB	20 Mbps	150 × 8 ÷ 20	60 s
600 MB	50 Mbps	600 × 8 ÷ 50	96 s
2000 MB	100 Mbps	2000 × 8 ÷ 100	160 s
4500 MB	200 Mbps	4500 × 8 ÷ 200	180 s

Why latency matters separately

A satellite link can have enormous bandwidth and still feel dreadful, because the signal must travel to orbit and back. Round trip times of roughly 600 ms make a video call painful even though large files transfer quickly.

ROUTE	TYPICAL ROUND TRIP
Within Lisbon	about 8 ms
Lisbon to London	about 40 ms
Lisbon to Sydney	about 320 ms
Via geostationary satellite	about 600 ms

For a video call or an online game, latency matters more than bandwidth. For downloading a large file, bandwidth matters more than latency.

KEYWORDS

bandwidth the maximum amount of data a connection can carry each second

latency the delay before data begins to arrive, measured as a round trip in milliseconds

bit rate the rate at which data is actually transferred at a given moment

Mbps megabits per second, the usual unit of connection speed

PRACTISE 3

1. Calculate the download time for a 250 MB file at 25 Mbps.
2. Calculate the download time for a 750 MB file at 100 Mbps.
3. A 50 MB file downloads in 4 seconds. What speed is the connection?
4. Explain why a satellite connection with high bandwidth is still poor for video calls.
5. Matilde says her 100 Mbps connection should download a 100 MB file in 1 second. Explain her mistake and give the correct time.

UNIT 7.3 · TOPIC 4

IP addresses

Every device on a network needs a unique **IP address**, so data can be delivered to it and to nothing else. It is the equivalent of a postal address.

IPv4

An IPv4 address is four numbers separated by full stops, such as `192.0.2.146`. Each number is one **octet**, stored in 8 bits, so it can be anything from 0 to 255.

Four octets of 8 bits gives 32 bits in total, which allows 2^{32} addresses, about 4.29 billion. That sounded limitless in the 1980s. With phones, laptops, televisions, watches and doorbells all online, it ran out.

IPv6

IPv6 uses 128 bits, written as eight groups of hexadecimal digits, such as `2001:0db8:85a3:0000:0000:8a2e:0370:7334`. That gives 2^{128} addresses, a number with 39 digits. It is comfortably enough to give every grain of sand on Earth many addresses of its own.

	IPV4	IPV6
Size	32 bits	128 bits
Written as	four numbers 0 to 255	eight groups of hexadecimal
Total addresses	about 4.29 billion	about 3.4×10^{38}
Example	`192.0.2.146`	`2001:db8::7334`

Static and dynamic

A **static** address never changes and suits a server, which must always be findable at the same place. A **dynamic** address is handed out temporarily by the router when a device joins and may differ tomorrow. Most home and school devices are dynamic, which conserves addresses.

KEYWORDS

IP address a unique number identifying a device on a network

octet one of the four numbers in an IPv4 address, from 0 to 255

static address an IP address that stays the same

dynamic address an IP address assigned temporarily when a device joins a network

PRACTISE 4

1. Explain why `192.0.2.300` cannot be a valid IPv4 address.
2. Calculate how many different values one octet can hold, and show your working.
3. Explain why IPv6 was needed.
4. Should a school's web server use a static or a dynamic address? Justify your answer.

URLs

A **URL**, or Uniform Resource Locator, is the full address of one particular resource on the web. Every part of it does a job.

Consider `https://www.primeschool.pt/computing/year7.html`

PART	EXAMPLE	WHAT IT DOES
Protocol	`https://`	the rules used, here secure web traffic
Subdomain	`www.`	which section of the domain to use
Domain name	`primeschool.pt`	the registered name of the organisation
Top-level domain	`.pt`	the category or country, here Portugal
Path	`/computing/`	the folder on the server
File	`year7.html`	the specific page requested

Reading a domain from the right

Domains are read right to left. In `www.primeschool.pt`, the `.pt` is decided first, then `primeschool` within it, then `www` within that. This matters for spotting fraud, as you will see in Topic 8.

Common top-level domains include `.pt` for Portugal, `.uk` for the United Kingdom, `.org` usually for organisations, `.edu` for education, and `.com` for commercial use.

- PRACTISE 5

1. Break `https://news.example.org/science/oceans/tides.html` into all six parts.
2. What does a `.pt` ending tell you, and what does it not tell you?
3. Explain the difference between a domain name and a URL.
4. Write a URL for a page called `timetable.html` in a folder called `pupils` on a site called `primeschool.pt`, using secure protocol.

UNIT 7.3 · TOPIC 6

The Domain Name System

Computers route data using IP addresses, but people remember names. The **Domain Name System**, or DNS, translates between the two. It is often called the phone book of the internet.

What happens when you press Enter

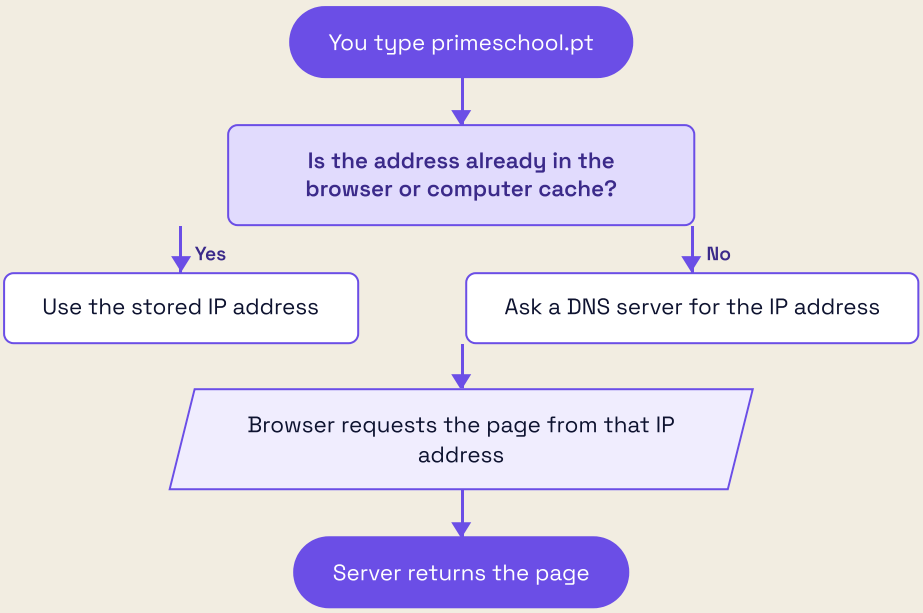


Figure 3.2 A DNS lookup. Caching the answer makes the next visit noticeably faster.

Caching

Once your computer learns an address it **caches** it, storing it for a while. The next visit skips the lookup entirely, which is one reason a site you use daily feels quicker than one you have never opened.

When DNS fails

If the DNS cannot find a name, the browser reports that the server could not be found. That usually means a typing mistake in the name, a site that no longer exists, or a problem with the DNS server itself rather than with the website.

✧ DID YOU KNOW?

The **404 error** is different and much more precise. A 404 means DNS worked perfectly, the server was found, and the server is telling you that the particular page you asked for does not exist on it. DNS failure means the building could not be located; 404 means the building is there but that room is not.

KEYWORDS

DNS the Domain Name System, which translates domain names into IP addresses

cache a temporary store of recently used data, kept to save fetching it again

404 error a message from a web server saying the requested page does not exist

PRACTISE 6

1. Explain in three steps what DNS does when you type a web address.
2. Explain why visiting a website a second time is often faster.
3. Distinguish between a DNS failure and a 404 error.
4. Suggest what you would check first if one website fails but every other site works normally.

UNIT 7.3 · TOPIC 7

Padlocks and HTTPS

HTTP is the protocol that carries web pages. **HTTPS** is the same protocol with **encryption** added. The s stands for secure.

What encryption does

Without HTTPS, data travels as readable text. Anyone able to observe the connection, for instance on an open café network, could read a password as it passes. With HTTPS, the data is scrambled using a key, so an observer sees only meaningless characters.

HTTPS provides three things:

- **Confidentiality.** Others cannot read what you send.
- **Integrity.** Others cannot alter it in transit without detection.
- **Authentication.** The site has a certificate proving it is the site the name refers to.

What the padlock does NOT mean

This is the part most people get wrong. The padlock means the **connection** is secure. It says nothing whatever about whether the **website** is honest.

A criminal can register a domain, obtain a certificate in minutes, and run a perfectly encrypted fraudulent shop. The padlock guarantees that nobody else is listening while you are being defrauded. It is necessary, but it is nowhere near sufficient.

THE PADLOCK PROMISES	THE PADLOCK DOES NOT PROMISE
your data is encrypted in transit	the company is genuine
the certificate matches the domain	the goods exist
nobody has altered the page en route	your data will be handled responsibly

KEYWORDS

HTTP the protocol used to transfer web pages

HTTPS HTTP with encryption, protecting data in transit

encryption scrambling data so only the intended recipient can read it

certificate a digital document proving a website belongs to the domain it claims

PRACTISE 7

1. Explain the difference between HTTP and HTTPS.
2. List the three protections HTTPS provides.
3. Duarte says a padlock means a shop is trustworthy. Explain carefully why he is mistaken.
4. Explain why HTTPS matters especially on a public café network.

UNIT 7.3 · TOPIC 8

Insecure and fraudulent websites

Since the padlock cannot tell you whether a site is honest, you need other checks.

Warning signs

- The domain is misspelt or has extra words: `primeschool-pt-login.example.com` is not `primeschool.pt`.
- The page demands urgent action: your account will close today, act now.
- It asks for information the organisation would never need, such as a full password by email.
- The writing has obvious spelling and grammar errors.
- The offer is implausible: a new phone for €20.
- Links in a message do not match the text shown. Hover to see the real destination.

Reading a domain safely

Remember that domains are read right to left, and the real domain is the part immediately before the top-level domain.

ADDRESS SHOWN	REAL DOMAIN	GENUINE?
`www.primeschool.pt`	`primeschool.pt`	yes
`primeschool.pt.login-secure.example.com`	`example.com`	no
`www.primeschoo1.pt`	`primeschoo1.pt`	no, the l is a digit 1

The second row is the classic trick. Everything before `example.com` is decoration chosen to reassure you.

Phishing

Phishing is a message pretending to be from someone you trust, designed to make you hand over information. The defence is simple and absolute: never follow a link in an unexpected message. Go to the site yourself by typing the address you already know.

If you think you have been caught, tell an adult immediately and change the password from a different device. Being embarrassed for five minutes is better than losing an account.

- PRACTISE 8

1. Identify the real domain in `secure.bank.pt.verify-account.example.net` and say whether it looks genuine.
2. List four warning signs of a fraudulent website.
3. A message says a parcel is held and asks for €2 to release it, with a link. Describe exactly what you should do.
4. Explain why "the site had a padlock" is not a defence against fraud.

UNIT 7.3 · TOPIC 9

Keeping it all secure

Passwords

A strong password is **long**, **unique** and **unguessable**. Length matters most: each extra character multiplies the work an attacker must do.

- Use a passphrase of several unrelated words, which is long yet memorable.
- Never reuse a password across sites. One breach then exposes everything.
- Never share a password, not even with a close friend.
- A password manager will remember unique passwords so you do not have to.

Two-factor authentication

Two-factor authentication, or 2FA, requires a second proof besides the password, usually a code from your phone. It means a stolen password alone is not enough to get in. Switch it on wherever it is offered, especially for email, because email can reset everything else.

Other sensible habits

HABIT	WHY IT MATTERS
Install updates promptly	most attacks exploit faults that were already fixed
Lock your screen when you walk away	physical access defeats most other protections
Be careful on public Wi-Fi	others may be able to observe an unencrypted connection
Think before you post	information helps attackers guess security answers
Check the address before typing a password	the commonest way people are caught

→ GO FURTHER: WHY LENGTH BEATS COMPLEXITY

A password of four random common words is far stronger than a short one full of symbols, because the number of possibilities grows with every character added. `Tram!7q` is short enough to attack by brute force; `sardine-lighthouse-copper-tuesday` is not, and you can actually remember it. Modern advice therefore favours long passphrases over unmemorable symbol soup.

• PRACTISE 9

1. Explain why reusing one password across many sites is dangerous.
2. Explain how two-factor authentication protects an account whose password has been stolen.
3. Suggest a memorable passphrase strategy, and explain why it is strong.
4. List three things you should never do on a public Wi-Fi network.

UNIT 7.3 · TOPIC 10

Searching well

A search engine visits pages continuously, stores what it finds in an index, and ranks the results when you ask a question. Learning to ask precisely saves a great deal of time.

Search operators

OPERATOR	EXAMPLE	EFFECT
`"quotes"`	`"pattern recognition"`	finds that exact phrase
`-word`	`python -snake`	excludes results containing that word
`site:`	`site:gov.pt rainfall`	searches only that site
`filetype:`	`filetype:pdf tram map`	finds only that file type
`OR`	`lisbon OR porto`	finds either term

Judging a source

Finding an answer is easy. Deciding whether to trust it is the real skill.

- **Who wrote it?** A named author or organisation with relevant expertise.
- **When?** Computing changes fast, so a page from 2011 may be badly out of date.
- **Why does it exist?** To inform, or to sell you something?
- **Can it be corroborated?** Two independent sources agreeing is far stronger than one.
- **Does it cite evidence?** Claims with sources beat confident assertions without any.

Never rely on a single source, and be especially careful with an answer that simply confirms what you already hoped was true.

KEYWORDS

search engine a system that indexes web pages and ranks them against a query

index the search engine's stored record of the pages it has visited

corroboration confirming information by checking an independent second source

PRACTISE 10

1. Write a search that finds PDF documents about rainfall on Portuguese government sites only.
2. Write a search for the exact phrase "undersea cable" that excludes results about television.
3. Evaluate a website of your choosing against the five questions above, writing one sentence for each.
4. Explain why two sources agreeing is stronger evidence than one source stating something confidently.

▲ CHALLENGE YOURSELF

Your family is choosing a broadband package for a flat in Cascais. Three are offered.

PACKAGE	BANDWIDTH	LATENCY	MONTHLY COST
Copper Basic	20 Mbps	25 ms	€22
Fibre Home	200 Mbps	8 ms	€35
Satellite Rural	100 Mbps	600 ms	€48

Answer all four parts, showing your working.

1. Calculate how long a 4500 MB film takes to download on each package.
2. Your sister plays online games where a delay above 100 ms makes play unfair. Which packages are usable, and why does bandwidth not rescue the others?
3. Two people stream video while a third downloads a 2000 MB update. Explain what happens to the bit rate each person experiences, and why it falls below the advertised bandwidth.
4. Recommend one package for a flat where two people work from home with daily video calls. Justify your choice using both bandwidth and latency, and say what you would give up.

★ FINAL PROJECT

A connection guide for Prime School families

The school wants a two-page guide helping families choose an internet connection and stay safe online. It must be accurate, genuinely useful, and written so that somebody who has never heard the word bandwidth can follow it.

THE BRIEF

Your guide must include:

- a plain-language explanation of bandwidth, latency and bit rate
- a comparison of at least three transmission methods
- a worked download-time calculation, with the working shown
- a table recommending a connection for three different households
- a diagram of what happens when someone types an address
- five safety rules, each with a one-sentence reason
- a clear explanation of what the padlock does and does not promise

MILESTONES

1. **Research**
   Find three real packages available in Portugal. Record the source and date for each.
2. **Calculate**
   Work out download times for a common file size on each. Show every step.
3. **Draw**
   Produce the DNS diagram using the correct flowchart symbols.
4. **Write**
   Draft the guide. Explain every technical term the first time you use it.
5. **Test it on a reader**
   Give it to somebody who does not study computing. Note every question they ask, then fix those parts.
6. **Evaluate**
   State which requirements are met, with evidence, plus one weakness and one improvement.

WORKED EXAMPLE TO CHECK AGAINST

```
File:  600 MB
Speed: 50 Mbps

600 MB x 8 = 4800 Mb
4800 / 50  = 96 seconds
           = 1 min 36 s

Same file at 10 Mbps:
4800 / 10  = 480 seconds
           = 8 minutes
```

The multiply by 8 is the step everybody forgets. File sizes are in bytes; connection speeds are in bits. Always convert before dividing.

HOW YOUR WORK WILL BE JUDGED

STRAND	SECURE
Accuracy	every technical statement is correct
Calculation	bits and bytes handled correctly, working shown
Diagram	DNS process drawn with correct symbols
Recommendations	justified using both bandwidth and latency
Safety	five rules, each with a real reason
Clarity	a non-specialist can follow it without help
Sources	research is cited with dates

UNIT 7.3

Evaluation and review

Think carefully about your work in this unit. For each statement, colour the circle that matches how confident you feel. Green means you could teach it to somebody else, amber means you can do it with your notes open, and red means you would like to go over it again.

explain the difference between the internet and the web	○	○	○
describe the chain from my device to a web server	○	○	○
explain why data travels in packets	○	○	○
compare copper, fibre and wireless transmission	○	○	○
define bandwidth, latency and bit rate, and tell them apart	○	○	○
calculate a download time, converting bytes to bits	○	○	○
explain what an IP address is and why IPv6 was needed	○	○	○
break a URL into its component parts	○	○	○
describe how DNS finds a web server	○	○	○
explain what HTTPS protects	○	○	○
explain what the padlock does not promise	○	○	○
read a domain from the right and spot a fake	○	○	○
recognise phishing and respond correctly	○	○	○
explain why passphrase length beats symbol complexity	○	○	○
search precisely and judge whether a source is reliable	○	○	○

WHAT CAN YOU DO?

- [ ] Separate the internet from the web
- [ ] Trace a request from desk to server
- [ ] Explain packets and why they help
- [ ] Choose a transmission method with reasons
- [ ] Distinguish bandwidth from latency
- [ ] Convert MB to Mb and calculate download times
- [ ] Explain IPv4, IPv6, static and dynamic
- [ ] Dissect any URL into its parts
- [ ] Describe a DNS lookup and caching
- [ ] Say exactly what HTTPS does and does not do
- [ ] Spot a fraudulent domain
- [ ] Respond correctly to a phishing message
- [ ] Build strong unique passphrases
- [ ] Use search operators precisely
- [ ] Judge a source before trusting it

UNIT

7.4

Managing data

✓ LEARNING OUTCOMES

In this unit you will learn to:

- ✓ explain what a data model is and why simplification is useful
- ✓ use a spreadsheet model to answer "what if" questions
- ✓ write formulae using cell references and the common functions
- ✓ tell relative and absolute cell referencing apart, and use each correctly
- ✓ describe a database in terms of tables, records, fields and primary keys
- ✓ choose sensible data types and field sizes for a database
- ✓ design a query that answers a specific question
- ✓ design a data collection form with validation and verification
- ✓ sort, filter and conditionally format data to reveal what matters
- ✓ choose the right chart, and the right software, for a given task

◆ GET STARTED

Should the tuck shop raise the price of a pastel de nata by ten cents?

Nobody can answer that by arguing. You need numbers. Discuss with a partner.

- What would you need to know before you could answer confidently?
- How could you work out the effect without actually changing the price and risking a bad month?
- What might the numbers fail to tell you about how pupils would react?

A **model** is a simplified version of something real, built so you can experiment safely. Change a number, and the model shows the consequences instantly. Nobody loses money while you find out.

In this unit you will build models in spreadsheets, organise information in databases, collect data without letting rubbish in, and present findings so a busy reader grasps them at a glance.

▮ KEYWORDS

data model a simplified representation of a real situation, built so it can be tested and explored

simulation running a model to see what would happen under particular conditions

what if question changing an input in a model to see the effect on the results

✦ WARM UP

The school runs a trip. The coach costs €250 for the day, and each pupil pays an entry fee of €4.50 on top of their share of the coach.

1. Work out the cost per pupil if 25 go.
2. Work out the cost per pupil if 50 go.
3. Explain why doubling the number of pupils does not halve the total cost per pupil.
4. Which single number would you most want to change to reduce the cost, and why?

↺ DO YOU REMEMBER?

You have the tools already.

- Unit 7.1: **data types** and why a phone number is text, not a number.
- Unit 7.1: **BIDMAS**, which spreadsheets obey exactly as Python does.
- Unit 7.3: judging whether a **source** is reliable, which matters just as much for data.

A spreadsheet formula is simply an expression, of the kind you already write in Python, attached to a cell.

UNIT 7.4 · TOPIC 1

Models, simulations and real scenarios

A model keeps the parts of a situation that matter and throws away the rest. A weather model ignores the colour of your coat. A tuck shop model ignores which pupil bought which biscuit.

Why models are worth building

BENEFIT	EXPLANATION
Safe	test a price rise without losing real money
Fast	try twenty possibilities in a minute
Cheap	no need to build anything physical
Repeatable	run the same scenario again and get the same answer
Dangerous situations	model a fire evacuation without a fire

The limits of a model

A model is only as good as its assumptions and its data. It cannot know what it was never told.

- The tuck shop model assumes pupils buy the same amount after a price rise. They may not.
- A traffic model assuming normal weather will be wrong during an Atlantic storm.
- A model built on last year's figures cannot know about this year's new café next door.

Always ask two questions of any model: what has it assumed, and where did the data come from?

✧ DID YOU KNOW?

Weather forecasting was the first great use of computer modelling. In 1922 Lewis Fry Richardson calculated a single six-hour forecast by hand. It took him six weeks, and the answer was wrong. His method, however, was right, and it is essentially what supercomputers run today, only rather more quickly.

• PRACTISE 1

1. Give three situations where modelling is safer than trying the real thing.
2. State two assumptions a model of the school canteen queue would have to make.
3. Explain why a model can be perfectly calculated and still give a misleading answer.
4. Suggest one thing a model of the tuck shop could never tell you.

UNIT 7.4 · TOPIC 2

Spreadsheets

A spreadsheet is a grid of **cells**. Each cell has an address made of its column letter and row number, such as `B4`. A cell holds a number, some text, or a **formula**.

Formulae always begin with =

A formula tells the spreadsheet to calculate rather than to display text. Referring to cells rather than typing numbers is the whole point: change the cell, and every formula updates.

FORMULA	MEANING
`=B2*C2`	multiply the value in B2 by the value in C2
`=B2+B3+B4`	add three cells
`=SUM(D2:D6)`	add everything from D2 to D6
`=AVERAGE(D2:D6)`	the mean of that range
`=MAX(D2:D6)`	the largest value
`=MIN(D2:D6)`	the smallest value
`=COUNT(D2:D6)`	how many cells hold numbers

A worked model: the school tuck shop

Here is one week of sales. Column D is not typed in: it is calculated by `=B2*C2`, filled down.

	A: ITEM	B: PRICE €	C: SOLD	D: INCOME €
2	Pastel de nata	1.20	148	177.60
3	Water 500 ml	0.80	210	168.00
4	Fruit bag	0.95	96	91.20
5	Cheese roll	1.60	132	211.20
6	Oat biscuit	0.70	174	121.80
7	**Total**		**760**	**769.80**

Check one row by hand: 1.20 × 148 = 177.60. The totals in row 7 use `=SUM(C2:C6)` and `=SUM(D2:D6)`.

The summary statistics come from the same range.

STATISTIC	FORMULA	RESULT
Total income	`=SUM(D2:D6)`	769.80
Mean income per line	`=AVERAGE(D2:D6)`	153.96
Best-selling line	`=MAX(D2:D6)`	211.20
Weakest line	`=MIN(D2:D6)`	91.20
Number of lines	`=COUNT(D2:D6)`	5

Notice something interesting. The cheese roll earns most money, at €211.20, yet water sells most units, at 210. Asking "which sells best" is not one question but two, and a good model answers both.

Relative and absolute referencing

When you copy `=B2*C2` from row 2 down to row 3, it becomes `=B3*C3`. The references moved with it. That is **relative referencing**, and it is usually exactly what you want.

Sometimes it is exactly what you do not want. Suppose cell `B1` holds the school fund percentage, 15%, and you want each line's contribution.

Writing `=D2*B1` in row 2 and copying it down gives `=D3*B2` in row 3, which points at the wrong cell and produces nonsense. The fix is a **absolute reference**: `$B$1`. The dollar signs lock the column and the row so they never shift.

ROW	FORMULA =D2*B1 COPIED DOWN	INCOME	15% SHARE
2	=D2*B1	177.60	26.64
3	=D3*B1	168.00	25.20
4	=D4*B1	91.20	13.68
5	=D5*B1	211.20	31.68
6	=D6*B1	121.80	18.27

The row number changes, as it should, but `$B$1` stays put. Total contribution to the school fund is €115.47, which is 15% of €769.80.

Asking a what if question

The model now earns its keep. Change the price of a pastel de nata in `B2` from 1.20 to 1.30 and every dependent cell updates instantly. If sales stayed at 148, income for that line would rise to €192.40, and the total would become €784.60.

That "if sales stayed the same" is the assumption doing the heavy lifting, and it is precisely the sort of assumption you must state out loud.

KEYWORDS

cell a single box in a spreadsheet, identified by column letter and row number

formula an instruction beginning with `=` that calculates a value

relative reference a reference that shifts when the formula is copied, such as `B2`

absolute reference a reference locked with dollar signs so it never shifts, such as `$B$1`

range a block of cells written as two corners, such as `D2:D6`

PRACTISE 2

1. Write the formula for the income of a line where the price is in `B9` and the quantity in `C9`.
2. Write the formula for the mean of `E2:E20`.
3. Explain what happens when `=A1*B1` is copied from row 1 to row 5, and give the resulting formula.
4. A VAT rate sits in `F1`. Write a formula for row 3 that can be safely copied down the column.
5. Using the tuck shop table, calculate by hand the income if oat biscuits rose to €0.85 and still sold 174. Show your working.

UNIT 7.4 · TOPIC 3

Databases

A spreadsheet is excellent for calculating. A **database** is better for storing large amounts of structured information and asking questions of it.

The vocabulary

TERM	MEANING	EXAMPLE
Table	a collection of data about one kind of thing	Pupils
Record	one row, all the data about one item	one pupil
Field	one column, one piece of information	Surname
Primary key	a field whose value is unique for every record	PupilID

A pupil table

PUPILID	SURNAME	FORENAME	YEAR	HOUSE
P7001	Almeida	Beatriz	7	Atlântico
P7002	Carvalho	Gonçalo	7	Tejo
P7003	Dias	Inês	7	Atlântico
P7004	Esteves	Tomás	8	Serra

There are four records and five fields. `PupilID` is the primary key.

Why a primary key must be unique

Two pupils can share a surname, a forename, a year and a house. If your key were `Surname`, the database could not tell two Almeidas apart, and updating one might change the other. A primary key guarantees that every record can be identified exactly.

This is also why a primary key should be something that never needs to change. A pupil's surname can change; their PupilID should not.

Field types and sizes

Choosing types carefully saves space and prevents errors.

FIELD	TYPE	WHY
PupilID	text, 5 characters	contains a letter, so not a number
Surname	text, 30 characters	long enough for real names
Year	integer	whole number, used for sorting
DateOfBirth	date	allows age calculations
BusPass	Boolean	yes or no

`Year` is a number because you may want to sort or filter by it. `PupilID` is text even though it contains digits, for the same reason a telephone number is text: you would never do arithmetic with it.

Queries

A **query** asks a question of the data and returns only the matching records.

QUESTION	CRITERIA
Which pupils are in Year 7?	`Year = 7`
Who is in Atlântico house?	`House = "Atlântico"`
Year 7 pupils in Atlântico?	`Year = 7 AND House = "Atlântico"`
Anyone in Year 7 or Year 8?	`Year = 7 OR Year = 8`
Anyone not in Year 7?	`Year <> 7`

Running the third query on the table above returns two records: Beatriz Almeida and Inês Dias.

KEYWORDS

database an organised collection of structured data

table a set of records about one kind of thing

record one complete entry, shown as a row

field one item of information, shown as a column

primary key a field that uniquely identifies every record

query a request that returns only the records matching stated criteria

PRACTISE 3

1. Design a table for the school library with at least five fields. State the primary key and justify it.
2. Give the type and a sensible size for each of your fields.
3. Write criteria for a query finding all books borrowed before a given date and not yet returned.
4. Explain why `Surname` would be a poor primary key.
5. How many records and how many fields are in the pupil table above?

UNIT 7.4 · TOPIC 4

Collecting data

A model built on bad data gives confident, precise, wrong answers. Collection is where quality is won or lost.

Validation: can this be right?

Validation is an automatic check the computer performs as data is entered. It cannot tell whether data is true, only whether it is possible.

CHECK	WHAT IT DOES	EXAMPLE
Range	value must fall between limits	Year between 7 and 13
Type	value must be the right data type	Quantity must be a whole number
Presence	the field cannot be left empty	Surname is required
Format	must match a pattern	postcode as `0000-000`
Length	a set number of characters	PupilID exactly 5 characters
Lookup	must be chosen from a list	House from a drop-down

Verification: is this what was meant?

Verification checks that data was entered accurately, usually by comparing two entries.

- **Double entry**: type a password twice and the computer compares them.
- **Visual check**: a human reads the entry against the original document.

The difference matters. If Gonçalo's year is typed as 8 instead of 7, validation is perfectly happy, because 8 is a legal year. Only verification catches it.

Designing a good form

- Ask only for what you genuinely need.
- Use drop-down lists rather than free text wherever the options are known.
- Make the required fields obvious before the user starts.
- Give a clear, specific error message: "Year must be between 7 and 13", not "invalid input".
- Never collect personal information without a good reason and permission.

→ GO FURTHER: GARBAGE IN, GARBAGE OUT

Programmers have a saying: garbage in, garbage out. A spreadsheet will happily calculate the mean of a column containing a mistyped 5000 that should have been 50, and will report the answer to two decimal places with total confidence. Precision is not the same thing as accuracy. Always look at your data before you trust a statistic drawn from it.

PRACTISE 4

1. Suggest a validation check for each: a pupil's age; an email address; a house name; a lunch order quantity.
2. Explain the difference between validation and verification, with an example that only verification would catch.
3. Design a form for a school trip booking. List every field, its type, and its validation check.
4. Explain why a drop-down list is usually better than a free text box.

UNIT 7.4 · TOPIC 5

Highlighting what matters

A table of 400 rows hides its meaning. These tools reveal it.

Sorting

Sorting reorders records by a field, ascending or descending. Sorting the tuck shop data by income descending puts the cheese roll first, immediately showing where the money comes from.

Always sort the whole table, never one column on its own. Sorting a single column detaches it from its row and destroys the data, silently.

Filtering

Filtering hides records that do not match a condition, without deleting anything. Filter the pupil table to `Year = 7` and only Year 7 records show. Remove the filter and everything returns.

Conditional formatting

Conditional formatting changes a cell's appearance based on its value, so patterns jump out.

RULE	EFFECT ON THE TUCK SHOP DATA
Income greater than 200, green	highlights the cheese roll
Income less than 100, red	highlights the fruit bag
Colour scale across D2:D6	shades every line from weakest to strongest

Choosing the right chart

CHART	BEST FOR	TUCK SHOP EXAMPLE
Bar or column	comparing separate categories	income by product
Line	change over time	daily sales across a term
Pie	parts of one whole	share of total income
Scatter	relationship between two variables	price against quantity sold

A pie chart of income shares would show the cheese roll at 27.4%, pastel de nata at 23.1%, water at 21.8%, oat biscuit at 15.8% and fruit bag at 11.8%.

Add those up and you get 99.9%, not 100%. Nothing is wrong: each figure was rounded to one decimal place, and the roundings do not perfectly cancel. Reporting this honestly is better practice than quietly adjusting a number to force a tidy total.

Rules for honest charts

- Label both axes, always, including units.
- Start a bar chart's value axis at zero. Starting elsewhere exaggerates differences dishonestly.
- Do not use a pie chart for more than about six categories, or for things that are not parts of one whole.
- Give the chart a title that states the finding, not just the topic.

PRACTISE 5

1. Choose the best chart type for: monthly rainfall in Sintra across a year; the share of pupils in each house; heights of pupils plotted against shoe size; income by tuck shop item.
2. Explain why sorting a single column is dangerous.
3. Describe a conditional formatting rule that would highlight every pupil in Year 7.
4. Explain why a bar chart starting at 60 rather than 0 can mislead a reader.
5. Explain why the five percentages above total 99.9%.

UNIT 7.4 · TOPIC 6

Choosing the right tool

Spreadsheets and databases overlap, and picking wrongly makes work far harder than it needs to be.

	SPREADSHEET	DATABASE
Best at	calculating, modelling, charting	storing and querying large structured data
Size	hundreds or thousands of rows	millions of records
Relationships	poor, data is repeated	strong, tables link together
Validation	limited	thorough, enforced by the design
Multiple users	awkward	designed for it
Learning	quick	slower

A simple test

- Will you mostly **calculate**? Choose a spreadsheet.
- Will you mostly **look things up** among many records? Choose a database.
- Is the same information typed in repeatedly on many rows? That repetition is the classic sign you need a database.

The tuck shop week, with five products, is clearly spreadsheet work. A record of every purchase by every pupil across a whole year is database work, because the pupil details would otherwise be repeated thousands of times.

PRACTISE 6

1. Recommend a tool for each, with a reason: modelling next year's trip costs; storing every library loan; a class's test results; the national census.
2. Explain what repeated data in a spreadsheet suggests, and why it causes problems.
3. Give one task a spreadsheet does better than a database, and one the database does better.

▲ CHALLENGE YOURSELF

The tuck shop must raise **€900** in a week to fund new sports equipment.

Using the week's figures on the previous pages, where total income was €769.80:

1. Calculate how much more is needed, and express it as a percentage increase on the current total. Give your answer to one decimal place.
2. If every price rose by 10% and quantities stayed the same, calculate the new total income. Would that alone meet the target?
3. If instead every quantity rose by 20% and prices stayed the same, calculate the new total. Compare the two strategies.
4. Write the formula you would put in a spreadsheet cell so the school fund percentage in `B1` can be changed once and update every row.
5. State two assumptions in your model, and explain how each might be wrong in real life.

★ FINAL PROJECT

The Prime School data study

Choose something real in your school, collect genuine data about it, model it, and present a finding somebody could act on. Anyone can make a chart. Your task is to produce a recommendation that stands up to questioning.

CHOOSE ONE

- tuck shop sales, and whether prices should change
- how pupils travel to school, and what would reduce car journeys
- library borrowing, and which subjects need more books
- lunch queue times, and how to shorten them
- classroom temperatures across the day, and where fans are needed

MILESTONES

1. **Ask a precise question**
   Not "look at the tuck shop" but "which two items should change price, and by how much".
2. **Design the collection**
   Build a form. State every field, its data type, and its validation rule.
3. **Collect real data**
   At least 20 records, genuinely gathered. Record when and how.
4. **Build the model**
   Use formulae with cell references, at least one absolute reference, and at least four functions.
5. **Reveal the pattern**
   Sort, filter, apply conditional formatting, and produce two charts of different types. Justify each choice.
6. **Answer the question**
   State your recommendation in one sentence, then support it with figures.
7. **Challenge yourself**
   List two assumptions and one way your data could be misleading.

WORKED EXAMPLE TO CHECK AGAINST

```
Income  =B2*C2    → 177.60
Total   =SUM(D2:D6)
        → 769.80
Mean    =AVERAGE(D2:D6)
        → 153.96
Best    =MAX(D2:D6)
        → 211.20
Fund    =D2*$B$1    (B1 = 15%)
        → 26.64
Total fund = 115.47
```

Every figure above was checked by hand. Do the same with yours: calculate one row manually and confirm the spreadsheet agrees before you trust the other nineteen.

HOW YOUR WORK WILL BE JUDGED

STRAND	SECURE
Question	precise enough to have a checkable answer
Collection	form has types and validation for every field
Data	20 or more genuine records, source recorded
Model	cell references used, absolute where needed, four functions
Presentation	two chart types, axes labelled, choices justified
Finding	a clear recommendation supported by figures
Honesty	assumptions and weaknesses stated openly

UNIT 7.4

Evaluation and review

Think carefully about your work in this unit. For each statement, colour the circle that matches how confident you feel. Green means you could teach it to somebody else, amber means you can do it with your notes open, and red means you would like to go over it again.

explain what a data model is and why it simplifies	○	○	○
state the assumptions a model depends on	○	○	○
write formulae using cell references	○	○	○
use SUM, AVERAGE, MAX, MIN and COUNT	○	○	○
explain relative and absolute referencing, and use $ correctly	○	○	○
answer a what if question using a spreadsheet model	○	○	○
define table, record, field and primary key	○	○	○
explain why a primary key must be unique and stable	○	○	○
choose sensible field types and sizes	○	○	○
write query criteria using AND, OR and not equal to	○	○	○
choose validation checks for a field	○	○	○
explain the difference between validation and verification	○	○	○
sort, filter and conditionally format safely	○	○	○
choose an appropriate chart and label it honestly	○	○	○
decide whether a task needs a spreadsheet or a database	○	○	○

✓ WHAT CAN YOU DO?

- ☐ Explain what a model is and what it ignores
- ☐ Question a model's assumptions
- ☐ Build a working spreadsheet model
- ☐ Use the five core functions confidently
- ☐ Lock a reference with $ when copying
- ☐ Run what if scenarios
- ☐ Describe a database in correct terms
- ☐ Pick a sound primary key
- ☐ Assign field types and sizes
- ☐ Write queries with multiple criteria
- ☐ Design a form that resists bad data
- ☐ Tell validation from verification
- ☐ Sort and filter without destroying data
- ☐ Choose and label a chart honestly
- ☐ Pick the right tool for the job

UNIT

7.5

Living with AI

Digital data

✓ LEARNING OUTCOMES

In this unit you will learn to:

- ✓ tell systems software and application software apart, and say what each is for
- ✓ describe where artificial intelligence already appears in daily life
- ✓ explain how a machine learning system is trained on example data
- ✓ design a simple rule-based AI system for a problem you choose
- ✓ recognise bias in training data and suggest how to reduce it
- ✓ explain how a bitmap image is stored using pixels and colour depth
- ✓ calculate the file size of a bitmap image
- ✓ compare bitmap and vector graphics, and choose between them
- ✓ explain how sound is sampled, and how text is stored using character sets
- ✓ use AND, OR and NOT gates, and complete a truth table

◆ GET STARTED

Your phone unlocked when it saw your face. How did it know it was you?

Think about the last hour of your day. Talk about these questions with a partner.

- Which devices recognised something: your face, your voice, your handwriting?
- Which app suggested something to you before you asked for it?
- Did any of those suggestions get it wrong? What happened?

None of those systems were given a rule that describes your face. Instead they were shown a great many examples until they could spot the pattern. That approach is called **machine learning**, and it is the engine behind most of what we now call artificial intelligence.

In this unit you will look inside these systems. You will see how a picture, a sound and a sentence all become numbers, because a computer can only ever work with numbers. Then you will design an AI system of your own, and think carefully about how such systems can go wrong.

▮ KEYWORDS

artificial intelligence computer systems that carry out tasks which normally need human intelligence, such as recognising a face or understanding speech

machine learning a way of building an AI system by training it on many examples rather than by writing a rule for every case

✦ WARM UP

Carolina is teaching a computer to tell a sardine from a sea bass using only photographs.

1. Write down four features the computer could measure from a photograph.
2. Carolina collects 300 photographs of sardines and 12 of sea bass. Explain why her system will probably work badly.
3. She takes every photograph at the fish market in Setúbal, on the same grey morning. Suggest one problem that might cause.
4. Suggest one situation where a wrong answer from this system would genuinely matter.

↺ DO YOU REMEMBER?

You have already met most of the building blocks.

- In Unit 7.1 you learned that everything a program stores has a **data type**.
- In Unit 7.3 you learned that data travels the internet as **bits**, the 1s and 0s.
- In Unit 7.4 you learned that a **model** is a simplified version of something real.

An AI system is a model built from data, running as software, on a machine that only understands bits. This unit joins those three ideas together.

UNIT 7.5 · TOPIC 1

Systems software and application software

All software falls into two families. Knowing which is which explains a great deal about how a computer behaves.

Systems software runs the machine itself. You rarely open it on purpose, but nothing works without it. **Application software** is the software you choose to run in order to get something done.

	SYSTEMS SOFTWARE	APPLICATION SOFTWARE
Purpose	keeps the computer running	performs a task for the user
Started by	the machine, automatically	you, deliberately
Examples	operating system, device drivers, utilities	browser, spreadsheet, IDLE, games
If it fails	the whole machine may stop	usually only that one program stops

What an operating system actually does

The **operating system**, such as Windows, macOS, Linux, Android or iOS, sits between your programs and the hardware. It has five main jobs.

- **Managing memory.** Deciding which program gets which part of the memory, and taking it back afterwards.
- **Managing processes.** Sharing the processor between programs so several appear to run at once.
- **Managing files.** Keeping track of where every file physically lives on the disc.
- **Managing devices.** Talking to the keyboard, screen, printer and network through drivers.
- **Managing users.** Handling logins, passwords and who is allowed to open what.

Utilities: the quiet helpers

A **utility** is a small systems program that maintains the computer: antivirus scanners, backup tools, disc clean-up, file compression. They are systems software because they look after the machine rather than doing your work for you.

KEYWORDS

systems software software that runs and maintains the computer itself

application software software that helps the user carry out a particular task

operating system the systems software that manages memory, processes, files, devices and users

utility a small systems program that maintains or protects the computer

PRACTISE 1

1. Sort these into systems or application software: a photo editor, a printer driver, an antivirus scanner, a web browser, Android, IDLE, a disc backup tool.
2. Explain why a device driver counts as systems software even though it controls a printer you chose to buy.
3. Tomás complains that his laptop slows down when he opens fifteen browser tabs. Which operating system job is under strain, and why?

UNIT 7.5 · TOPIC 2

AI around us

Artificial intelligence is not one technology. It is a family of techniques that let a machine do something we would call intelligent if a person did it.

Where you already meet it

SYSTEM	WHAT IT DOES	HOW IT LEARNED
Face unlock	recognises your face in any light	trained on many images of faces
Voice assistant	turns speech into words, then into an action	trained on recordings of many speakers
Video suggestions	predicts what you will watch next	learns from what millions of people watched
Spam filtering	separates junk from real messages	learns from messages people marked as junk
Bus arrival prediction	estimates when the 728 will reach your stop	learns from thousands of past journeys

Two ways to build an intelligent system

A **rule-based system** follows rules a person wrote. If the temperature rises above 30 degrees and the humidity is below 20 per cent, raise the fire risk. Rules are easy to explain and easy to correct, but somebody must think of every case in advance.

A **machine learning system** is shown labelled examples and works out its own rules. Show it ten thousand photographs, each labelled sardine or sea bass, and it discovers which combinations of pixels matter. Nobody writes the rule, and often nobody can say precisely what the rule became.

How training works

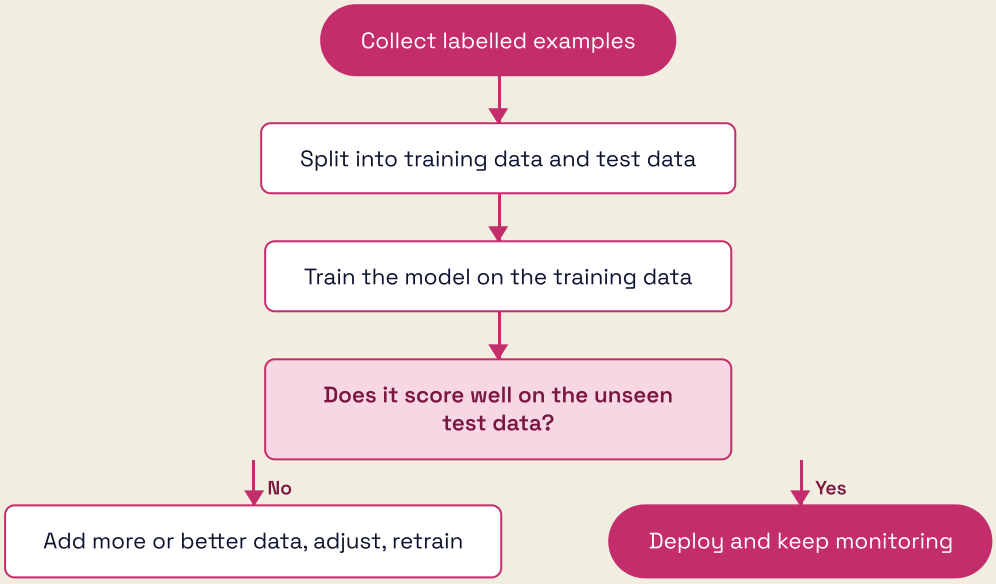


Figure 5.1 The training loop. The test data must stay unseen, otherwise a good score proves nothing.

Keeping test data separate is essential. A model that has already seen the answers can score perfectly and still be useless on anything new, rather like a pupil who memorised last year's paper.

When AI gets it wrong

An AI system learns from the data it is given, including the data's faults. This is called **bias**.

- A voice system trained mostly on adult male speakers may struggle with a child's voice.
- A system trained only on photographs taken in bright sunshine will fail on a grey day.
- A model trained on last year's data will not know about anything that changed this year.

Bias is rarely deliberate. It usually comes from data that was easy to collect rather than data that was representative.

✧ DID YOU KNOW?

In 1950 Alan Turing suggested a test: if a person holding a typed conversation cannot tell whether they are talking to a machine or a human, the machine should count as intelligent. Turing called it the imitation game. Seventy-five years later, people still argue about whether passing it really proves anything at all.

KEYWORDS

rule-based system an AI system that follows rules written by a person

training data the labelled examples used to teach a machine learning model

test data examples kept back from training, used to check whether the model really learned

bias unfairness in a system's results, usually caused by unrepresentative training data

PRACTISE 2

1. For each system, say whether rules or machine learning suits it better, and why: a chess timer; recognising handwriting; deciding whether a pupil is late; suggesting a song.
2. A school builds an AI that predicts which pupils need extra help, trained only on data from Year 11. Give two reasons the predictions may be poor for Year 7.
3. Explain why a model that scores 100 per cent on its training data might still be a bad model.
4. Suggest three ways to make the sardine and sea bass dataset from the Warm up more representative.

UNIT 7.5 · TOPIC 3

Designing your own AI system

Designing an AI system is mostly about asking good questions before writing any code.

The five questions

1. **What decision must it make?** State it in one sentence, with the possible answers listed.
2. **What data would help?** Which measurements actually relate to the decision?
3. **Where will the data come from?** Who collects it, how often, and is that fair to those involved?
4. **How will you know it works?** Decide the test before you build the thing.
5. **What happens when it is wrong?** Every system is wrong sometimes. Plan for it.

A worked design: the beach flag adviser

Cascais Council wants a system that suggests which flag to fly at a beach: green for safe, yellow for caution, red for no swimming.

The decision. Given today's conditions, output green, yellow or red.

The data. Wave height in metres, wind speed in km/h, water temperature, number of lifeguards on duty, and whether a storm warning is in force.

A rule-based first attempt.

```
def flag_colour(wave_m, wind_kmh, lifeguards):
    if wave_m ⩾ 2.5 or wind_kmh ⩾ 45:
        return "Red"
    elif wave_m ⩾ 1.2 or wind_kmh ⩾ 25:
        return "Yellow"
    elif lifeguards == 0:
        return "Yellow"
    else:
        return "Green"

print(flag_colour(0.6, 12, 2))
print(flag_colour(1.4, 18, 2))
print(flag_colour(2.9, 30, 3))
print(flag_colour(0.5, 10, 0))
```

```
Green
Yellow
Red
Yellow
```

Trace the fourth call. The waves and wind are both calm, so the first two conditions are false, but there are no lifeguards, so the system sensibly advises caution.

Testing it. Boundary values matter most, because that is where rules disagree.

TEST	WAVE (M)	WIND (KM/H)	LIFEGUARDS	EXPECTED	WHY
1	1.2	10	2	Yellow	exactly on the yellow boundary
2	1.1	10	2	Green	just below the yellow boundary
3	2.5	10	2	Red	exactly on the red boundary
4	0.4	45	2	Red	wind alone can force red

When it is wrong. A flag that is too cautious costs the town visitors. A flag that is too relaxed risks a life. The two errors are not equally serious, so the system should lean towards caution, and a lifeguard must always be able to

overrule it.

→ GO FURTHER: FROM RULES TO LEARNING

The rule-based adviser works, but it can only ever be as good as the numbers somebody chose. A machine learning version would take ten years of records showing the conditions each day and the flag an experienced lifeguard actually flew, then learn the boundaries from real decisions.

That approach captures judgement nobody wrote down, but it brings two costs. It needs a great deal of reliable historical data, and it becomes much harder to explain why a particular flag was chosen. In safety work, being able to explain a decision often matters more than being slightly more accurate.

• PRACTISE 3

1. Work through the five questions for a system that decides whether the school field is too wet for games.
2. Write the rule-based function for your system in Python, then test it with at least four values including two boundaries.
3. Describe one situation where your system being wrong would cause a real problem, and how you would reduce the harm.

UNIT 7.5 · TOPIC 4

Representing images

A computer stores no colours and no pictures. It stores numbers. A **bitmap** image is a grid of tiny squares called **pixels**, and each pixel is stored as a number that stands for a colour.

Resolution and colour depth

Resolution is how many pixels the image contains, given as width times height. **Colour depth** is how many bits are used for each pixel. More bits means more available colours.

COLOUR DEPTH	COLOURS AVAILABLE	WORKING
1 bit	2	2^1
2 bits	4	2^2
4 bits	16	2^4
8 bits	256	2^8
24 bits	16 777 216	2^{24}, eight bits each for red, green and blue

Twenty-four bit colour is called true colour, because it holds more shades than the human eye can distinguish.

Calculating a file size

The rule is short: **file size in bits = width × height × colour depth**.

A photograph of the Belém Tower is 800 pixels wide, 600 pixels tall, in 24-bit colour.

- Pixels: 800 × 600 = 480 000
- Bits: 480 000 × 24 = 11 520 000
- Bytes: 11 520 000 ÷ 8 = 1 440 000
- Kilobytes: 1 440 000 ÷ 1000 = 1440 kB
- Megabytes: 1440 ÷ 1000 = 1.44 MB

Doubling the resolution in both directions makes the image four times larger, because you double the width and the height.

The trade-off

CHOICE	ADVANTAGE	DISADVANTAGE
Higher resolution	more detail, prints larger	bigger file, slower to send
Lower resolution	small file, quick to load	looks blocky when enlarged
Greater colour depth	smoother shading	bigger file
Lower colour depth	small file	banding in skies and skin tones

KEYWORDS

pixel the smallest single element of a bitmap image

resolution the number of pixels in an image, given as width by height

colour depth the number of bits used to store the colour of one pixel

bitmap an image stored as a grid of pixels

PRACTISE 4

1. Calculate the file size in kilobytes of a 400 × 300 image at 8-bit colour.
2. Calculate the file size in megabytes of a 1920 × 1080 image at 24-bit colour.
3. Inês halves both the width and the height of a photograph. By what factor does the file size change? Explain.
4. Explain why a photograph of a sunset looks worse at 4-bit colour than a black and white line drawing does.

UNIT 7.5 · TOPIC 5

Vector graphics

A **vector graphic** stores instructions rather than pixels. Instead of recording six hundred thousand coloured squares, it records something closer to: a blue circle, centred here, with this radius, and a red line from here to there.

Why that matters

Because the image is a set of instructions, the computer redraws it at whatever size you ask for. A vector logo is equally crisp on a business card and on the side of a building. Enlarge a bitmap and you simply get bigger squares.

	BITMAP	VECTOR
Stores	a colour for every pixel	shapes and their properties
Enlarging	becomes blocky	stays perfectly sharp
File size	grows with resolution	grows with number of shapes
Best for	photographs	logos, maps, diagrams, icons
Editing	change pixels	move or restyle whole objects

What a vector file records

For each object the file stores its type and its properties: the coordinates, the dimensions, the fill colour, the line colour and the line thickness. A simple flag might be three rectangles, each with a position, a size and a fill colour.

Vectors are unsuitable for photographs. A photograph of the Douro valley has no clean shapes to describe, just millions of subtly different pixels, so a bitmap is the sensible choice.

DID YOU KNOW?

The Prime School logo on the cover of this book is a vector graphic. That single file is used for the badge on a blazer, the sign at the school gate and the icon in a browser tab. If it were a bitmap, the school would need a separate file for every size, and the gate sign would look like a mosaic.

PRACTISE 5

1. Choose bitmap or vector for each, with a reason: a class photograph; the school crest; a map of the school grounds; a scan of a handwritten letter; an app icon.
2. Explain why enlarging a bitmap makes it blurry but enlarging a vector does not.
3. List the properties a vector file would need to store in order to draw a yellow circle with a black outline.
4. Rodrigo says vector files are always smaller. Give one example where that is false, and explain why.

UNIT 7.5 · TOPIC 6

Representing sound and text

Sound: taking samples

Sound reaches your ear as a continuous wave. A computer cannot store something continuous, so it measures the wave's height many times per second. Each measurement is a **sample**.

- **Sample rate** is how many samples are taken per second, measured in hertz. Music on a CD uses 44 100 Hz.
- **Bit depth** is how many bits store each sample. CDs use 16 bits.

The rule mirrors the image one: **file size in bits = sample rate × bit depth × seconds × channels**.

A 30 second mono recording of the sea at Guincho, at 44 100 Hz and 16 bits:

- Samples: 44 100 × 30 = 1 323 000
- Bits: 1 323 000 × 16 = 21 168 000
- Bytes: 21 168 000 ÷ 8 = 2 646 000
- Megabytes: 2 646 000 ÷ 1 000 000 = 2.646 MB

In stereo it would be two channels, so 5.292 MB.

A higher sample rate captures higher frequencies and sounds closer to the original. A lower rate saves space but makes music sound dull and muffled.

Text: character sets

Text is stored by giving every character a number, using an agreed **character set**.

ASCII uses 7 bits, giving 128 codes: the English letters, digits, punctuation and some control codes. It has no room for á, ç, ã or õ, which makes it useless for writing Portuguese.

Unicode was created to fix exactly that. It has codes for over 149 000 characters, covering essentially every writing system, plus emoji. Portuguese, Greek, Japanese and Arabic all fit in the same document.

CHARACTER	ASCII CODE	CHARACTER	ASCII CODE
A	65	a	97
B	66	b	98
Z	90	z	122
0	48	space	32

Notice that the codes run in order. `B` is one more than `A`, and every lower case letter is exactly 32 more than its capital. That regularity lets a program sort words alphabetically with simple arithmetic.

```
print(ord("A"), ord("a"))
print(chr(67), chr(99))
print(ord("a") - ord("A"))
```

```
65 97
C c
32
```

KEYWORDS

sample a single measurement of a sound wave's height

sample rate the number of samples taken each second, measured in hertz

bit depth the number of bits used to store one sample

character set an agreed list matching every character to a number

ASCII a 7-bit character set with 128 codes, covering English only

Unicode a character set covering the writing systems of the world

PRACTISE 6

1. Calculate the size in megabytes of a 60 second mono recording at 22 050 Hz and 8 bits.
2. Calculate the size in megabytes of the same recording in stereo at 44 100 Hz and 16 bits.
3. Use `ord()` to find the codes for `M` and `m`, and check the difference is 32.
4. Explain why a Portuguese school register could not safely be stored in ASCII.
5. A sound file is halved in size by changing one setting. Suggest two different settings that would achieve this, and describe how each affects quality.

UNIT 7.5 · TOPIC 7

Introduction to logic gates

Underneath everything, a computer is millions of tiny switches. A **logic gate** is a circuit that takes one or two inputs, each 1 or 0, and produces a single output. Three gates are enough to build a computer.

The three gates

AND outputs 1 only when both inputs are 1. Think of two switches in a row: the current needs both closed.

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

OR outputs 1 when at least one input is 1.

A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1

NOT takes a single input and reverses it.

A	NOT A
0	1
1	0

Combining gates

Real circuits chain gates together. Consider a greenhouse fan in the Alentejo that should run only when it is hot **and** the door is **not** open.

```
Fan = Hot AND (NOT DoorOpen)
```

HOT	DOOROPEN	NOT DOOROPEN	FAN
0	0	1	0
0	1	0	0
1	0	1	1
1	1	0	0

The fan runs in exactly one of the four situations, the third row, which is precisely what was asked for.

The same logic in Python

The comparison operators from Unit 7.1 combine with `and`, `or` and `not` in exactly this way.

```
def fan_on(hot, door_open):
    return hot and not door_open

print(fan_on(True, False))
print(fan_on(True, True))
print(fan_on(False, False))
```

```
True
False
False
```

KEYWORDS

logic gate a circuit that produces an output of 1 or 0 from its inputs

truth table a table listing every possible combination of inputs and the resulting output

AND gate outputs 1 only when both inputs are 1

OR gate outputs 1 when at least one input is 1

NOT gate reverses its single input

PRACTISE 7

1. Copy and complete a truth table for `A OR (NOT B)`.
2. A door alarm should sound when the door is open AND the alarm is armed. Write the expression and its truth table.
3. Write the Python function for question 2 and test all four combinations.
4. Explain why a truth table for three inputs needs eight rows.

▲ CHALLENGE YOURSELF

Build a **museum climate monitor** for the Oceanário.

A tank room must trigger an alert when the water is too warm **or** too cold, but never while maintenance mode is switched on. The safe range is 18 °C to 24 °C inclusive.

1. Write the logic expression using AND, OR and NOT.
2. Build the full truth table. Use TooWarm, TooCold and Maintenance as your three inputs, so your table needs eight rows.
3. Write a Python function `alert(temp, maintenance)` that takes the actual temperature and returns `True` or `False`.
4. Test it against this plan and confirm every row.

TEST	TEMPERATURE	MAINTENANCE	EXPECTED
1	21.0	False	False
2	17.9	False	True
3	24.0	False	False
4	24.1	False	True
5	30.0	True	False

★ FINAL PROJECT

An honest AI for the school library

The librarian wants a system that recommends a next book to each pupil. She is enthusiastic but wary: she has read about recommendation systems that trap people in a narrow rut, and she refuses to install anything she cannot explain to a parent. Design it properly.

THE BRIEF

Your design document must:

- state in one sentence what decision the system makes
- list the data it would use, and say where each item comes from
- decide whether rules or machine learning suits it better, and justify the choice
- include a rule-based version written as a Python function
- include a test plan of at least six rows with boundary cases
- identify two ways the system could become biased, with a fix for each
- explain in plain language how a pupil could find out why a book was suggested

MILESTONES

1. **Frame the problem**
   Answer the five design questions from Topic 3 in full sentences.
2. **Design the data**
   List every field, its data type, and how it would be collected fairly.
3. **Write the rules**
   Build your Python function. Keep it readable and comment the thresholds.
4. **Test it**
   Run every row of your plan, including boundaries, and record real results.
5. **Interrogate it**
   Write half a page on bias, on who could be badly served, and on what you would monitor after launch.

A STARTING POINT

```
Pupil: Leonor, Year 7
Last 3 books: 2 adventure, 1
history
Average pages read: 210
Reading level: secure

Suggested: adventure, 180-260
pages
Reason shown to pupil:
  "You finished 2 adventure books
   of a similar length."
```

Notice that the reason is part of the output, not an afterthought. A system that cannot explain itself cannot be corrected.

HOW YOUR WORK WILL BE JUDGED

STRAND	SECURE
Framing	the decision and its answers are stated precisely
Data	fields are sensible, typed, and fairly sourced
Reasoning	the rules versus learning choice is genuinely justified
Program	function runs and returns correct results
Testing	six or more rows, boundaries included, real results recorded
Ethics	two credible biases identified with workable fixes
Explainability	a pupil could understand why they got that suggestion

UNIT 7.5

Evaluation and review

Think carefully about your work in this unit. For each statement, colour the circle that matches how confident you feel. Green means you could teach it to somebody else, amber means you can do it with your notes open, and red means you would like to go over it again.

I CAN ...	GREEN	AMBER	RED
tell systems software and application software apart	○	○	○
describe the five jobs of an operating system	○	○	○
give examples of AI in everyday life and say how each learned	○	○	○
explain the difference between a rule-based and a learning system	○	○	○
explain why test data must be kept separate from training data	○	○	○
recognise bias in a dataset and suggest how to reduce it	○	○	○
design a simple AI system using the five design questions	○	○	○
explain pixels, resolution and colour depth	○	○	○
calculate the file size of a bitmap image	○	○	○
compare bitmap and vector graphics and choose between them	○	○	○
explain sampling, and calculate the size of a sound file	○	○	○
explain why Unicode replaced ASCII for most purposes	○	○	○
complete truth tables for AND, OR and NOT, and combinations	○	○	○

✓ WHAT CAN YOU DO?

- ☐ Classify any software as systems or application
- ☐ Describe what an operating system manages
- ☐ Spot AI at work in everyday devices
- ☐ Explain how a model is trained and tested
- ☐ Identify bias and suggest practical fixes
- ☐ Design a rule-based system from scratch
- ☐ Calculate bitmap file sizes confidently
- ☐ Choose between bitmap and vector
- ☐ Calculate sound file sizes from sample rate and bit depth
- ☐ Explain ASCII, Unicode and why accents matter
- ☐ Build and read a truth table
- ☐ Write logic conditions in Python

UNIT

7.6

Sequencing and pattern recognition

Getting the message across

✓ LEARNING OUTCOMES

In this unit you will learn to:

- ✓ recognise patterns in problems and use them to simplify a solution
- ✓ replace repeated code with a count-controlled loop
- ✓ use a list to store many related values under one name
- ✓ spot the patterns shared by different text-based programming languages
- ✓ write a project plan with clear, ordered, achievable tasks
- ✓ build a test plan and record real results against it
- ✓ identify syntax, runtime and logic errors, and debug them methodically
- ✓ program a sequence of lights with precise timing
- ✓ control the brightness of individual LEDs
- ✓ combine sequence, selection and iteration in one working program

◆ GET STARTED

A lighthouse on the Atlantic coast never speaks, yet every sailor knows exactly which one it is.

The lighthouse at Cabo da Roca flashes in a fixed rhythm, and no two lighthouses nearby share the same pattern. Discuss with a partner.

- How can a light with only two states, on and off, carry useful information?
- Why must the timing be exact rather than roughly right?
- What other everyday signals use a pattern of light or sound to mean something?

A pattern is information. Once you can see the pattern in a problem, you can usually replace a long list of instructions with a short, elegant one. That is the heart of this unit.

You will learn to spot repetition, to express it as a loop, and to build precisely timed sequences of light. Then you will plan, build, test and evaluate a complete signalling project of your own.

▮ KEYWORDS

pattern recognition noticing similarities or repetition within a problem, and using them to simplify the solution

iteration repeating a set of instructions, also called looping

sequence instructions carried out one after another in a fixed order

✦ WARM UP

Look at each sequence and work out the rule, then give the next two terms.

1. 2, 4, 8, 16, 32, ...
2. 1, 4, 9, 16, 25, ...
3. on, on, off, on, on, off, on, ...
4. 5, 10, 20, 40, ...

Now the important question. For sequence 1, would you rather write out the first thirty terms by hand, or write one rule that produces them? Explain what that tells you about programming.

↺ DO YOU REMEMBER?

Everything in this unit builds on tools you already have.

- **Sequence** from Unit 7.1: steps in order.
- **Selection** from Unit 7.1: `if`, `elif` and `else`.
- **The micro:bit display** from Unit 7.2: `display.set_pixel(x, y, brightness)`.
- **Test plans** from Units 7.1 and 7.2: normal, boundary and erroneous data.

The one genuinely new idea is **iteration**: getting the computer to repeat something for you.

UNIT 7.6 · TOPIC 1

Pattern recognition

Pattern recognition means spotting what repeats. It is what turns a tedious problem into a short one.

Seeing the repetition

Suppose you want to light the top row of the micro:bit display. Without pattern recognition you would write five near-identical lines.

```
from microbit import *

display.set_pixel(0, 0, 9)
display.set_pixel(1, 0, 9)
display.set_pixel(2, 0, 9)
display.set_pixel(3, 0, 9)
display.set_pixel(4, 0, 9)
```

Look carefully at what changes and what stays the same. Only the first number changes, and it counts 0, 1, 2, 3, 4. Everything else is identical. That is a pattern, and a **for loop** expresses it in two lines.

```
from microbit import *

for x in range(5):
    display.set_pixel(x, 0, 9)
```

Understanding range()

`range()` produces a sequence of whole numbers. It is the engine of most loops you will write.

CALL	PRODUCES	COUNT
`range(5)`	0, 1, 2, 3, 4	5 values
`range(1, 6)`	1, 2, 3, 4, 5	5 values
`range(0, 10, 2)`	0, 2, 4, 6, 8	5 values
`range(5, 0, -1)`	5, 4, 3, 2, 1	5 values

The count always **starts at the first number and stops before the last**. `range(5)` gives five values ending at 4, not at 5. This catches everybody once.

```
for i in range(1, 4):
    print("Beep", i)
print("Done")
```

```
Beep 1
Beep 2
Beep 3
Done
```

Why loops matter so much

- A loop of three lines can do the work of three hundred.
- Changing the behaviour means editing one line, not three hundred.
- There is one place for a bug to hide instead of three hundred.

KEYWORDS

for loop a count-controlled loop that repeats a fixed number of times

range() a command that produces a sequence of whole numbers for a loop

count-controlled loop a loop that runs a known number of times

PRACTISE 1

1. Write a loop that prints the numbers 1 to 10.
2. Write a loop that prints the 7 times table up to 7 × 12.
3. Predict the output of `for i in range(3, 9, 2): print(i)`, then check it.
4. Rewrite these three lines as a loop: `display.set_pixel(2,0,9)`, `display.set_pixel(2,1,9)`, `display.set_pixel(2,2,9)`.
5. Explain why `range(5)` does not include 5.

UNIT 7.6 · TOPIC 2

Nested loops and lists

A loop inside a loop

To light the whole display you need every combination of `x` and `y`. Put one loop inside another, and the inner loop runs completely for each pass of the outer one.

```
from microbit import *

for y in range(5):
    for x in range(5):
        display.set_pixel(x, y, 9)
        sleep(100)
```

The outer loop runs 5 times, the inner loop 5 times each, so `set_pixel` is called 5 × 5 = 25 times, lighting the display one LED at a time from the top left.

Lists: many values, one name

A **list** stores a collection of values in order, under a single name. Square brackets create one, and an **index** picks a value out. Indexes start at 0.

```
routes = ["15E", "18E", "24E", "28E"]
print(routes[0])
print(routes[3])
print(len(routes))
```

```
15E
28E
4
```

The last item of a four-item list is at index 3, because counting starts at 0. Asking for `routes[4]` produces an `IndexError`.

Looping through a list

```
delays = [100, 200, 400, 800]
total = 0
for d in delays:
    total = total + d
print("Total wait:", total, "ms")
```

```
Total wait: 1500 ms
```

Check it: 100 + 200 + 400 + 800 = 1500. A list plus a loop lets one short program handle a sequence of any length.

✧ DID YOU KNOW?

The lighthouse at Cabo da Roca, the westernmost point of mainland Europe, flashes four times every seventeen seconds. Every major lighthouse has its own timing, published in charts, so a navigator can identify the coast by counting flashes and timing the gap. It is a system of sequencing that predates computers by well over a century.

■ KEYWORDS

list an ordered collection of values stored under one name

index the position of an item in a list, counting from 0

nested loop a loop placed inside another loop

• PRACTISE 2

1. Create a list of five Portuguese cities and print the second and the last.
2. Write a loop that prints every item of your list on its own line.
3. Use a nested loop to light only the outer border of the display.
4. A list has 7 items. What is the index of the last one? What error appears if you ask for index 7?
5. Write a loop that adds up the list `[12, 8, 21, 4]` and prints the total. Check the answer by hand.

UNIT 7.6 · TOPIC 3

Patterns between programming languages

Once you know one text-based language, the next is far easier, because the same handful of ideas appears everywhere in slightly different clothing.

IDEA	PYTHON	JAVASCRIPT	WHAT IS THE SAME
Output	`print("Hi")`	`console.log("Hi")`	a command plus a value
Variable	`total = 0`	`let total = 0;`	a name, then a value
Condition	`if x > 5:`	`if (x > 5) {`	the same comparison operators
Count loop	`for i in range(5):`	`for (let i=0; i<5; i++) {`	start, limit, step
Comment	`# note`	`// note`	ignored by the computer

What actually differs

- **Blocks.** Python uses indentation. Most other languages use curly brackets `{ }`.
- **Line endings.** Many languages need a semicolon at the end of each statement. Python does not.
- **Declaring variables.** Some languages make you announce a variable, and its type, before use.

What never differs

Every text-based language has sequence, selection and iteration. Every one has variables, data types and operators. Every one needs testing and debugging. Learning your second language is mostly learning new punctuation for ideas you already hold.

→ GO FURTHER: PSEUDOCODE

Pseudocode describes an algorithm in structured English, with no language's punctuation at all. It lets you plan without deciding on a language yet.

```
BEGIN
  SET count TO 0
  FOR each flash FROM 1 TO 4
    TURN light ON
    WAIT 500 milliseconds
    TURN light OFF
    WAIT 500 milliseconds
    SET count TO count + 1
  END FOR
  DISPLAY count
END
```

Any programmer can read that, whichever language they use. Examiners like it for the same reason.

• PRACTISE 3

1. Write in pseudocode an algorithm that counts down from 10 and then displays a message.
2. Look at the JavaScript column. Write down two differences from Python you would need to remember.
3. Explain why learning a second programming language is usually much quicker than learning the first.

UNIT 7.6 · TOPIC 4

Project plans and test plans

Bigger programs need planning before typing. Two documents do that work.

The project plan

A **project plan** lists the tasks in order, with a realistic estimate for each. A good task is small enough to finish in one sitting and specific enough that you know when it is done.

#	TASK	ESTIMATE	DEPENDS ON
1	Agree the flash pattern and timings	20 min	nothing
2	Draw the flowchart	25 min	task 1
3	Write the test plan	20 min	task 1
4	Code a single flash	20 min	task 2
5	Add the loop and the gap	25 min	task 4
6	Add the button to change pattern	30 min	task 5
7	Run every test and record results	30 min	tasks 3 and 6
8	Write the evaluation	25 min	task 7

Notice that the test plan is written at task 3, **before** the code exists. Deciding what correct looks like before you build is what keeps you honest.

The test plan

TEST	TYPE	INPUT	EXPECTED	ACTUAL
1	Normal	press A once	4 flashes, then a 3 s gap	
2	Normal	leave alone 30 s	pattern repeats, timing steady	
3	Boundary	press A during a flash	current cycle finishes cleanly	
4	Boundary	press A and B together	switches to pattern 2	
5	Erroneous	press A ten times quickly	no crash, no queue of patterns	

Leave the **Actual** column blank until you run it, then write what genuinely happened, not what you hoped would happen.

KEYWORDS

project plan an ordered list of tasks with estimates, used to organise a project

dependency a task that must be finished before another can begin

test plan a prepared table of inputs and expected outputs, written before the code

PRACTISE 4

1. Write a project plan of at least six tasks for a device that signals SOS.
2. Mark the dependencies. Which tasks could two people do at the same time?
3. Write a five-row test plan for the same device, before writing any code.
4. Explain why writing the test plan first makes your testing more trustworthy.

UNIT 7.6 · TOPIC 5

Identifying errors and debugging

You met the three families of error in Unit 7.1. Loops introduce a fourth trap that is worth naming separately.

TYPE	WHAT HAPPENS	TYPICAL CAUSE
Syntax	nothing runs	missing colon, bracket or quote
Runtime	stops part way	index out of range, wrong type
Logic	runs, wrong answer	< instead of ⩽, wrong operator
Off-by-one	runs, one too many or too few	misreading how `range()` counts

The off-by-one error

```
for i in range(1, 5):
    print("Flash", i)
```

```
Flash 1
Flash 2
Flash 3
Flash 4
```

If you wanted five flashes, this gives four. `range(1, 5)` stops **before** 5. Write `range(1, 6)`, or the clearer `range(5)` if the number itself does not matter.

Reading a runtime error

```
Traceback (most recent call last):
  File "signal.py", line 4, in <module>
    print(routes[4])
IndexError: list index out of range
```

The list has four items at indexes 0 to 3, so index 4 does not exist. Read from the bottom line upwards: the error type, then the line, then the file.

A debugging routine

1. Read the error from the bottom up.
2. Check the line named, and the line above it.
3. Check colons, brackets, quotes and capital letters.
4. Check every indentation.
5. For a loop, print the counter each pass to see what it really does.
6. Change one thing, then test again.

```
for i in range(3):
    print("i is now", i)
```

```
i is now 0
i is now 1
i is now 2
```

Printing the counter takes ten seconds and settles most loop arguments instantly.

KEYWORDS

off-by-one error a loop that runs one time too many or too few, usually a `range()` mistake

IndexError a runtime error caused by asking for a list position that does not exist

traceback the error report Python prints, read from the bottom upwards

---

PRACTISE 5

1. `for i in range(0, 10, 3)` runs how many times, and what are the values? Check by running it.
2. Find the fault: a program should flash 6 times but uses `range(1, 6)`. Correct it two different ways.
3. A list of 5 items gives an `IndexError` at index 5. Explain why, and give the valid range of indexes.
4. Write a loop with a deliberate off-by-one error, then use a print statement to expose it.

Sequences and lights

Now the ideas combine. A signalling device is a precisely timed sequence of light.

One flash, then many

```
from microbit import *

for flash in range(4):
    display.show(Image.SQUARE)
    sleep(500)
    display.clear()
    sleep(500)
sleep(3000)
```

Each flash takes 500 + 500 = 1000 ms, so four flashes take 4000 ms. Adding the 3000 ms gap makes a full cycle of 7000 ms, exactly 7 seconds.

Timing from a list

Storing the timings in a list makes the pattern easy to change, which is exactly what a good design allows.

```
from microbit import *

pattern = [200, 200, 600, 600, 200, 200]
for gap in pattern:
    display.show(Image.SQUARE)
    sleep(gap)
    display.clear()
    sleep(200)
```

Six flashes: three short, one long pair, then two short. Total lit time is 200 + 200 + 600 + 600 + 200 + 200 = 2000 ms, and the six gaps of 200 ms add 1200 ms, giving a cycle of 3200 ms.

Morse code: a real sequencing problem

Morse represents each letter as dots and dashes. A dash lasts three times a dot.

LETTER	MORSE	LETTER	MORSE
S	...	O	- - -
E	.	T	-
A	.-	N	-.

With a dot of 200 ms, SOS is three dots, three dashes, three dots.

- Dots: 6 × 200 = 1200 ms
- Dashes: 3 × 600 = 1800 ms
- Total lit time: 3000 ms

```
from microbit import *

DOT = 200
DASH = 600

def signal(times):
    for t in times:
        display.show(Image.SQUARE)
        sleep(t)
        display.clear()
        sleep(DOT)

sos = [DOT, DOT, DOT, DASH, DASH, DASH, DOT, DOT, DOT]
while True:
    signal(sos)
    sleep(2000)
```

Writing `DOT` and `DASH` as named values at the top means the whole message can be sped up or slowed down by editing two numbers.

- PRACTISE 6

1. Write a program that flashes 3 times with 300 ms on and 300 ms off, then waits 2 seconds.
2. Calculate the total cycle time of your answer to question 1.
3. Write the list for the letter A, then for the word TEA, using the table above.
4. Change the SOS program so the whole message runs at half speed, editing only two lines.

UNIT 7.6 · TOPIC 7

Light brightness

Each LED has ten brightness levels, from 0 (off) to 9 (full). Brightness turns a simple flash into something far more expressive.

Fading in and out

```
from microbit import *

while True:
    for b in range(10):
        display.set_pixel(2, 2, b)
        sleep(80)
    for b in range(9, -1, -1):
        display.set_pixel(2, 2, b)
        sleep(80)
```

The first loop counts 0 up to 9, brightening the centre LED. The second counts 9 down to 0, dimming it. `range(9, -1, -1)` stops before −1, so it does include 0.

Each loop takes 10 × 80 = 800 ms, giving a smooth 1600 ms fade cycle.

A brightness gradient

```
from microbit import *

for x in range(5):
    for y in range(5):
        display.set_pixel(x, y, x * 2)
```

Column 0 gets brightness 0, column 1 gets 2, then 4, 6 and 8. The display shows a gradient from dark on the left to bright on the right. Multiplying by 2 keeps every value inside the legal range of 0 to 9, since the largest is 4 × 2 = 8.

KEYWORDS

brightness how strongly an LED is lit, from 0 to 9 on the micro:bit

gradient a smooth change in brightness across a display

PRACTISE 7

1. Write a program that fades the whole display up and down together.
2. Create a gradient that runs top to bottom instead of left to right.
3. Explain why `x * 2` is safe but `x * 3` would fail for the last column.
4. Write a program where button A brightens the centre LED and button B dims it, stopping sensibly at 0 and 9.

▲ CHALLENGE YOURSELF

Build a **harbour signalling system** for Cascais marina.

The harbour master needs one device showing three states, and the timings must be exact.

STATE	PATTERN	MEANING
Clear	steady, brightness 4	safe to enter
Caution	1 flash per second, brightness 9	proceed slowly
Closed	3 rapid flashes, then a 2 s pause	do not enter

Requirements:

1. Button A moves to the next state, cycling clear, caution, closed, clear.
2. Store the three states in a list and use the index to select the current one.
3. Write a function `cycle_ms(state)` returning the length of one full cycle in milliseconds.
4. For caution use 500 ms on and 500 ms off. For closed use 150 ms on and 150 ms off, three times, then the 2000 ms pause.

Prove `cycle_ms` with this plan.

TEST	STATE	EXPECTED CYCLE	WORKING
1	caution	1000 ms	500 + 500
2	closed	2900 ms	3 × (150 + 150) + 2000
3	clear	0 ms	steady, so no cycle

★ FINAL PROJECT

Getting the message across

Design and build a device that communicates a message using only light. It must use a sequence, a loop and a decision, and its timings must be exact enough that somebody else can read the message without being told what it says.

CHOOSE ONE

- a lighthouse with an identifying flash pattern of your own design
- a Morse code sender for a short word
- a countdown signal for the start of a swimming race
- a corridor status light for the science laboratory

MILESTONES

1. **Design the pattern**
   Write the exact timings in a table. Every value in milliseconds, nothing left vague.
2. **Write the project plan**
   Six or more ordered tasks with estimates and dependencies.
3. **Write the test plan first**
   Five or more rows, with the Actual column left empty.
4. **Build it**
   Use a list for the timings and a loop to play them. Name your constants at the top.
5. **Add a decision**
   A button must change the pattern, or a sensor must alter it.
6. **Test and record**
   Fill in the Actual column honestly. Time one full cycle with a stopwatch and compare it against your calculation.
7. **Evaluate**
   Each requirement met or not, with evidence, plus one weakness and one measurable improvement.

TIMING WORKED EXAMPLE

```
Pattern: 4 flashes, then a gap

on  500 ms  x4 = 2000 ms
off 500 ms  x4 = 2000 ms
gap        = 3000 ms
----------------------
one cycle  = 7000 ms
in 60 s: 60000 / 7000
         = 8 full cycles
           (4000 ms left over)
```

Always calculate the cycle before you build, then check it with a stopwatch afterwards. If the two disagree, one of them is a bug worth finding.

HOW YOUR WORK WILL BE JUDGED

STRAND	SECURE
Pattern design	timings stated exactly, in a table, in milliseconds
Planning	six or more ordered tasks with dependencies
Iteration	a loop replaces repeated code; a list holds the timings
Selection	a button or sensor genuinely changes behaviour
Testing	plan written first, Actual column completed honestly
Accuracy	measured cycle matches the calculated cycle
Evaluation	evidenced, with a real weakness and a measurable fix

UNIT 7.6

Evaluation and review

Think carefully about your work in this unit. For each statement, colour the circle that matches how confident you feel. Green means you could teach it to somebody else, amber means you can do it with your notes open, and red means you would like to go over it again.

I CAN …	GREEN	AMBER	RED
spot a repeating pattern in a problem	○	○	○
replace repeated lines with a for loop	○	○	○
predict exactly what range() will produce	○	○	○
use a nested loop to cover a two-dimensional grid	○	○	○
store values in a list and read them by index	○	○	○
loop through a list to process every item	○	○	○
recognise the shared patterns between languages	○	○	○
write an algorithm in pseudocode	○	○	○
write a project plan with tasks, estimates and dependencies	○	○	○
write a test plan before writing the code	○	○	○
recognise and fix an off-by-one error	○	○	○
read a traceback and debug methodically	○	○	○
build a precisely timed sequence of lights	○	○	○
calculate the total cycle time of a pattern	○	○	○
control the brightness of individual LEDs	○	○	○

WHAT CAN YOU DO?

- ☐ Recognise repetition and remove it
- ☐ Write count-controlled loops with confidence
- ☐ Use range() with a start, stop and step
- ☐ Nest one loop inside another
- ☐ Create lists and access items by index
- ☐ Loop over a list to total or process it
- ☐ Read simple code in another language
- ☐ Plan algorithms in pseudocode
- ☐ Plan a project into ordered tasks
- ☐ Write the test plan before the code
- ☐ Avoid and fix off-by-one errors
- ☐ Build exact timed light sequences
- ☐ Calculate a full cycle in milliseconds
- ☐ Fade and grade LED brightness

Glossary

Every keyword from the six units, in alphabetical order.

404 error a message from a web server saying the requested page does not exist

absolute reference a reference locked with dollar signs so it never shifts, such as `$B$1`

algorithm a precise, ordered set of steps that solves a problem

AND gate outputs 1 only when both inputs are 1

application software software that helps the user carry out a particular task

artificial intelligence computer systems that carry out tasks which normally need human intelligence, such as recognising a face or understanding speech

ASCII a 7-bit character set with 128 codes, covering English only

assignment using `=` to put a value into a variable

backbone the very high capacity cables that link cities and continents

bandwidth the maximum amount of data a connection can carry each second

bias unfairness in a system's results, usually caused by unrepresentative training data

binary a number system using only the digits 0 and 1, used to represent all data in a computer

bit a single binary digit, either 1 or 0

bit depth the number of bits used to store one sample

bit rate the rate at which data is actually transferred at a given moment

bitmap an image stored as a grid of pixels

block-based programming programming by dragging ready-made blocks together, as in Scratch

Boolean a value that is either `True` or `False`

boundary data values at the very edge of a decision, where behaviour changes

brightness how strongly an LED is lit, from 0 to 9 on the micro:bit

byte a group of eight bits

cache a temporary store of recently used data, kept to save fetching it again

casting converting a value from one data type to another, for example `int("20")`

cell a single box in a spreadsheet, identified by column letter and row number

certificate a digital document proving a website belongs to the domain it claims

character set an agreed list matching every character to a number

colour depth the number of bits used to store the colour of one pixel

comment a note in the code, starting with `#`, that Python ignores

concatenation joining two pieces of text end to end with `+`

condition an expression that is either True or False

conditional formatting changing how a cell looks according to the value it holds

corroboration confirming information by checking an independent second source

count-controlled loop a loop that runs a known number of times

data model a simplified representation of a real situation, built so it can be tested and explored

data type the kind of value being stored, such as string, integer, real or Boolean

database an organised collection of structured data

debugging finding and correcting errors in a program

decomposition breaking a large problem into smaller problems that can each be solved separately

dependency a task that must be finished before another can begin

DNS the Domain Name System, which translates domain names into IP addresses

dynamic address an IP address assigned temporarily when a device joins a network

elif short for else if, a further condition tested when earlier ones are false

embedded system a computer built into a device to control it, rather than a general-purpose computer

encryption scrambling data so only the intended recipient can read it

erroneous data values that should be rejected or handled safely

field one item of information, shown as a column

filter hiding records that do not match a condition, without deleting them

firmware systems software stored permanently in a device to start and control it

flowchart a diagram that shows an algorithm using standard symbols

for loop a count-controlled loop that repeats a fixed number of times

formula an instruction beginning with `=` that calculates a value

gradient a smooth change in brightness across a display

indentation the spaces at the start of a line that show which block a statement belongs to

index the search engine's stored record of the pages it has visited

IndexError a runtime error caused by asking for a list position that does not exist

infinite loop a loop such as `while True:` that repeats until the device is switched off

input data given to a program by the user or by a sensor

integer a whole number, positive, negative or zero

internet the global network of connected networks that carries data between devices

IP address a unique number identifying a device on a network

ISP Internet Service Provider, the company connecting you to the internet

iteration repeating a set of instructions, also called looping

latency the delay before data begins to arrive, measured as a round trip in milliseconds

LED a small light, twenty-five of which form the micro:bit display

list an ordered collection of values stored under one name

logic error a fault that lets the program run but produces the wrong result

logic gate a circuit that produces an output of 1 or 0 from its inputs

machine learning a way of building an AI system by training it on many examples rather than by writing a rule for every case

Mbps megabits per second, the usual unit of connection speed

micro:bit a small programmable board with a display, buttons and built-in sensors

millisecond one thousandth of a second, the unit used by `sleep()`

modulus the remainder after a division, written % in Python

nested loop a loop placed inside another loop

normal data typical values the program will usually meet

NOT gate reverses its single input

octet one of the four numbers in an IPv4 address, from 0 to 255

off-by-one error a loop that runs one time too many or too few, usually a `range()` mistake

operating system the systems software that manages memory, processes, files, devices and users

OR gate outputs 1 when at least one input is 1

output information the program sends out, usually to the screen

packet a small block of data with an address, part of a larger transmission

pattern recognition noticing similarities or repetition within a problem, and using them to simplify the solution

phishing a message pretending to be from a trusted sender, designed to steal information

pixel the smallest single element of a bitmap image

primary key a field that uniquely identifies every record

project plan an ordered list of tasks with estimates, used to organise a project

protocol an agreed set of rules that lets different systems communicate

pseudocode an algorithm written in structured English rather than a real language

Python a widely used text-based programming language, popular in schools, science and industry

query a request that returns only the records matching stated criteria

range a block of cells written as two corners, such as `D2:D6`

range() a command that produces a sequence of whole numbers for a loop

real a number with a decimal part, called a `float` in Python

record one complete entry, shown as a row

relative reference a reference that shifts when the formula is copied, such as `B2`

resolution the number of pixels in an image, given as width by height

router a device that directs data between networks and to the correct device

rule-based system an AI system that follows rules written by a person

runtime error an error that appears while the program is running

sample a single measurement of a sound wave's height

sample rate the number of samples taken each second, measured in hertz

Scratch a block-based programming language often used before moving to text

script mode writing a complete program in a file so it can be saved and run again

search engine a system that indexes web pages and ranks them against a query

selection choosing between different paths through a program depending on a condition

sensor a component that measures something physical and reports it as a number

sequence program steps carried out one after another in a fixed order

shell a window that runs one instruction at a time and shows the result straight away

simulation running a model to see what would happen under particular conditions

sort reordering records by the values in a chosen field

spreadsheet a grid of cells used to store data and calculate with formulae

static address an IP address that stays the same

string a sequence of characters treated as text

sub-routine a named block of code that can be called whenever it is needed

syntax error a mistake in the rules of the language that stops the program running

systems software software that runs and maintains the computer itself

table a set of records about one kind of thing

test data examples kept back from training, used to check whether the model really learned

test plan a prepared table of inputs and expected outputs used to check a program

text-based programming writing a program by typing instructions as words and symbols rather than dragging ready-made blocks

threshold a value used as the dividing line in a decision

traceback the error report Python prints, read from the bottom upwards

training data the labelled examples used to teach a machine learning model

truth table a table listing every possible combination of inputs and the resulting output

two-factor authentication a second proof of identity, besides a password, needed to sign in

Unicode a character set covering the writing systems of the world

URL the full address of one resource on the web

utility a small systems program that maintains or protects the computer

validation an automatic check that entered data is possible and sensible

variable a named place in memory that holds a value which can change while the program runs

vector graphic an image stored as shapes and their properties rather than as pixels

verification a check that data was entered accurately, often by entering it twice

what if question changing an input in a model to see the effect on the results

Wi-Fi a wireless connection using radio waves within a building

World Wide Web the collection of linked pages and files that travels over the internet

PRIME BOOKS

Computing & Robotics

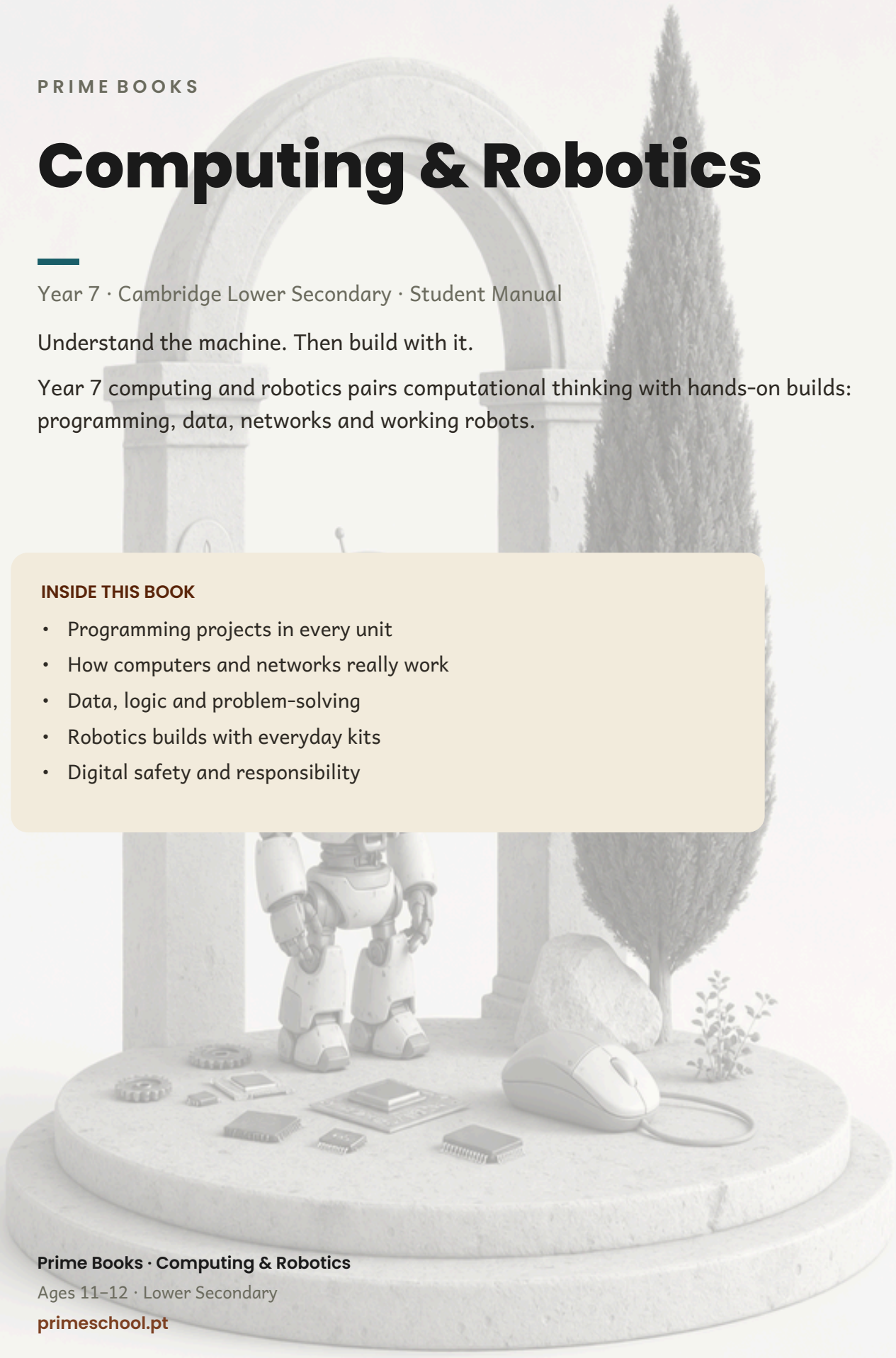
Year 7 · Cambridge Lower Secondary · Student Manual

Understand the machine. Then build with it.

Year 7 computing and robotics pairs computational thinking with hands-on builds: programming, data, networks and working robots.

INSIDE THIS BOOK

- Programming projects in every unit
- How computers and networks really work
- Data, logic and problem-solving
- Robotics builds with everyday kits
- Digital safety and responsibility



Prime Books · Computing & Robotics

Ages 11–12 · Lower Secondary

primeschool.pt